

Revisiting Parameter-Based Knowledge Editing in Large Language Models: Theoretical Limits and Empirical Evidence

Wanying Ren^{*1} Xin Song^{*1} Futing Wang² Guoxiu He¹ Aixin Sun³

Abstract

Parameter-based knowledge editing updates the internal knowledge of large language models (LLMs) via localized weight modifications and has attracted significant attention. However, most existing methods overlook fundamental theoretical limitations and are rarely evaluated under realistic, practice-oriented settings. In this paper, we first present a theoretical analysis based on the dimensional Collapse Hypothesis, explaining how localized parameter edits can propagate along fragile directions in the representation space, inducing global interference and ultimately causing reasoning collapse. Building on this insight, we conduct a comprehensive empirical evaluation by systematically varying knowledge complexity, number of edits, evaluation dimensions, and baseline methods. Our results show that parameter-based editing methods consistently damage core LLM capabilities. In contrast, a simple retrieval-based baseline achieves consistently stronger performance than all parameter-editing methods across all evaluated conditions. These findings highlight that preserving the fundamental capabilities of LLMs after knowledge editing should be a central concern for future research.

1. Introduction

Large language models (LLMs) (Zeng et al., 2023; Touvron et al., 2023; OpenAI, 2023) acquire extensive world knowledge (Jiang et al., 2020; AlKhamissi et al., 2022; Zhang et al., 2023b) and remarkable contextual reasoning abilities (Liu et al., 2023; Lee et al.) through large-scale pre-training and post-training (Brown et al., 2020; Ouyang et al., 2022).

^{*}Equal contribution ¹East China Normal University, Shanghai, China ²Westlake University, Hangzhou, China ³Nanyang Technological University, Singapore. Correspondence to: Guoxiu He <gxhe@fem.ecnu.edu.cn>.

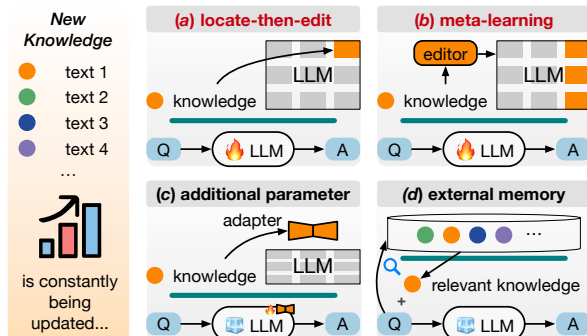


Figure 1. Training and inference workflows for four types of knowledge editing methods. The training process is shown above the green line, while the inference stage is shown below. (a) and (b) perform knowledge editing by modifying the LLM parameters.

However, as world knowledge continuously evolves, some information encoded in LLMs inevitably becomes outdated or inaccurate (Mousavi et al., 2024; Ji et al., 2023). To address this issue, knowledge editing methods (Sinitisin et al., 2020; Rawat et al., 2021) have been proposed to incorporate newly acquired knowledge into LLMs through minimal parameter modifications or external knowledge storage.

Knowledge editing methods can be broadly categorized into four main types, distinguished by how they adjust LLM parameters to incorporate collected textual knowledge. Figure 1 illustrates their respective workflows during training and inference. (a) **Locate-then-edit** methods (Meng et al., 2022b; Meng et al.; Li et al., 2024c) assume that specific knowledge is associated with certain LLM parameters; they first *locate* the neurons corresponding to the target knowledge and then *edit* the relevant parameters to incorporate the new knowledge. (b) **Meta-learning**-based methods (De Cao et al., 2021; Tan et al.) assume that the patterns of parameter changes during knowledge updates can be learned; an editor model is trained to modify the parameters in specific layers of the LLM associated with the target knowledge. (c) **Additional parameter**-based methods (Huang et al., 2023; Yu et al., 2024; Wang et al., 2024b) assume that new knowledge can be stored in additional parameters external to the LLM; adapter layers or other auxiliary components are introduced to encode new knowledge while leaving the base LLM parameters unchanged. (d) **External memory**-based

methods (Hartvigsen et al., 2024; Mitchell et al., 2022) store updated knowledge as texts, embeddings, hidden states, or lightweight models; during inference, the most relevant information is retrieved to support LLM reasoning without modifying LLM parameters.

Methods that directly modify the internal parameters of LLMs have attracted substantial research attention, partly because they appear more principled, incur no additional inference-time overhead compared to the original model, and often achieve seemingly perfect knowledge updates under specific evaluation protocols. **From a theoretical perspective**, however, we argue that these advantages can be misleading. Although localized parameter edits may be effective for isolated factual updates, dimensional collapse in LLM representation spaces creates fragile low-variance directions where even small weight perturbations are disproportionately amplified. This amplification distorts representation geometry and propagates through subsequent layers, leading to catastrophic forgetting of unrelated knowledge, degraded generalization, and impaired reasoning. In this work, we develop a theoretical analysis in Section 4 to characterize the fundamental limitations of parameter-based knowledge editing methods. **From an empirical perspective**, most existing studies have not fully evaluated knowledge editing methods in terms of LLMs’ core capabilities and realistic application scenarios. While some work has highlighted deficiencies in specific evaluation dimensions, these discussions remain fragmented and have yet to form a widely accepted, systematic framework. LLMs encode not only isolated factual knowledge (*e.g.*, triples) but also complex event-level knowledge, and their capabilities extend beyond answering simple facts to flexible knowledge application and deeper reasoning, such as reverse relation inference and multi-hop reasoning. Realistic editing scenarios should involve sequential, continuous updates rather than single-shot edits, and parameter-based editing methods should be systematically compared with external-knowledge-based paradigms (*e.g.*, retrieval-augmented generation (RAG)). Empirical results are shown in Section 5.

Our contributions are threefold ¹:

- (1) We develop a geometric theoretical framework that characterizes how localized parameter edits destabilize LLM representations.
- (2) We perform a comprehensive empirical evaluation under realistic settings, showing that such edits consistently degrade core LLM capabilities.
- (3) Our results clarify the fundamental limitations of parameter-based knowledge editing and highlight the stability–efficiency trade-off in the mechanisms.

¹The code is available at [GitHub Link](#).

2. Related Work

Knowledge editing in LLMs has received increasing attention, with numerous methods aimed at improving accuracy and efficiency (Li & Chu, 2025; Fang et al., 2025b; Scialanga et al., 2025; Nie et al., 2025; Zhang et al., 2026; Liu et al.). While most work focuses on improving factual correctness for isolated updates, a few studies have also highlighted potential limitations of parameter-based editing.

For example, Pinter & Elhadad (2023) noted conflicts between optimizing factual consistency and pre-training objectives, while Wang et al. (2025) quantified the *ripple effect* of local edits on hidden representations, leading to degraded model performance. Similarly, Gu et al. (2024b); Xie et al. (2025) observed that improving factual accuracy often undermines reasoning, natural language inference, and question answering, with edits to as little as 1% of parameters harming unrelated tasks. Furthermore, minor parameter modifications can reduce text coherence (Yang et al., 2024a), trigger model collapse in some specific methods such as ROME (Yang et al., 2024b), amplify existing biases (Halevy et al., 2024) or yield misleading evaluations under teacher-forcing settings (Yang et al., 2025). Mechanistic analyses have highlighted failure modes in locate-and-edit methods, offering theoretical insights into key-value representations (Yan et al., 2024). A broader discussion appears in Appendix A.1.

While these studies have significantly enhanced empirical understanding of knowledge editing risks, their contributions remain largely phenomenological and scoped to specific phenomena: individual failure patterns, method variants, or isolated evaluation settings. They have not yet converged into a unified theoretical framework that explains why parameter edits can broadly compromise LLM performance. Nor have they established comprehensive and consistent evaluation standards (as summarized in Table 4 in Appendix A.2) that capture core LLM capabilities and realistic application scenarios.

In contrast, this paper makes two complementary contributions, balancing theoretical analysis and empirical evaluation. First, we develop a mathematically grounded framework based on the dimensional Collapse Hypothesis, formally demonstrate that even minor parameter edits disrupt pre-training-induced knowledge geometry, resulting in systemic reasoning failures. Second, We conduct a comprehensive benchmark comparing parameter-based and external-knowledge-based paradigms (*e.g.*, RAG) across diverse knowledge formats, reasoning-oriented LLMs, multi-step editing scenarios, and practical inference tasks. This evaluation incorporates extensive metrics for robustness and capability preservation, surpassing prior benchmarks (Chen et al., 2025). Together, these contributions establish both theoretical underpinnings and empirical evidence for the limitations of parameter-based knowledge editing, uncovering

potential oversights in critical application-relevant characteristics and providing practical implications and actionable directions for future research.

3. Preliminaries

Triplet-based knowledge editing focuses on updating a factual triple $e = (h, r, t)$ to a new triple $e' = (h, r, t_*)$ within an original language model f^0 . For example, editing (*Barack Obama, born in, Hawaii*) to (*Barack Obama, born in, Kenya*). The updated model f^1 is obtained by applying the editor E to the initial model f^0 and the knowledge update e' :

$$f^1 = \mathcal{E}(f^0, e'), \quad \text{such that } e' \in f^1 \quad (1)$$

Factual knowledge, such as triples, can be represented as a single prompt–answer pair $e = [(x, y)]$, whereas more complex event is represented using multiple prompt–answer pairs $e = [(x_1, y_1), \dots, (x_m, y_m)]$, where x denotes the prompt to elicit the knowledge, y represents the answer, and m denotes the number of pairs. After editing, the updated model f^1 should output the ground truth y for the corresponding prompts x .

Editing Scenarios. **Single Editing** refers to the process where the LLM updates only one piece of knowledge ($E = [e_1]$) at a time. **Sequential Editing** also known as lifelong editing or continual editing (Hartvigsen et al., 2024; Yu et al., 2024; Wang et al., 2024b), refers to repeatedly applying the editing method to update a sequence of knowledge $E_t = [e_1, e_2, \dots, e_t]$. Consequently, the LLM evolves from f^0 to f^t over multiple updates. Note that, in the sequential editing setting, each method operates differently: **locate-then-edit** methods modify a subset of the LLM’s parameters with each update; **meta-learning**-based methods use a learned editor to update the parameters of the target model; **additional parameter**-based methods repeatedly update external parameter components; **external memory**-based methods only update an external knowledge base, such as a text corpus or vector store, that stores newly introduced knowledge.

Evaluation Dimensions. After t updates, the edited model f^t is evaluated across four dimensions (Zhang et al., 2024b):

- **Reliability:** The edited LLM should reliably produce the updated target outputs for the prompts in E_t , ensuring the persistence of the edits:

$$\mathbb{E}_{(x_i, y_i) \sim E_t} \mathbb{I}\{\arg \max_y f^t(y | x_i) = y_i\}. \quad (2)$$

- **Generalization:** The edited LLM should extend beyond the exact edits and correctly respond to paraphrased prompts,

denoted as $N(E_t)$:

$$\mathbb{E}_{(x_i, y_i) \sim N(E_t)} \mathbb{I}\{\arg \max_y f^t(y | x_i) = y_i\}. \quad (3)$$

- **Locality:** The edited LLM should retain its original behavior for queries unrelated to the edits, denoted $O(E_t)$, ensuring stability and minimal unintended changes:

$$\mathbb{E}_{(x_i, y_i) \sim O(E_t)} \mathbb{I}\{f^t(y | x_i) = f^0(y | x_i)\}. \quad (4)$$

- **Portability:** The edited LLM should propagate the edits effectively, enabling downstream reasoning $D(E_t)$, including handling aliases, causal relationships, and reverse relations. Despite its importance, this dimension has been overlooked in many studies (shown in Table 4):

$$\mathbb{E}_{(x_i, y_i) \sim D(E_t)} \mathbb{I}\{\arg \max_y f^t(y | x_i) = y_i\}. \quad (5)$$

Evaluation for General Tasks. In addition to the knowledge editing evaluation dimensions, it is also important to assess whether the edited LLM retains its ability to handle general tasks. In this study, mathematical reasoning datasets are utilized to evaluate the edited model’s reasoning capabilities. A successful editing process achieves precise targeted updates while preserving the LLM’s core capabilities, including generalization, reasoning, and performance on unrelated knowledge.

4. Theoretical Analysis

This section develops a geometric framework for understanding why parameter-based knowledge editing can induce disproportionate side effects in large language models. Our analysis is motivated by a simple observation: if hidden representations are dimensionally collapsed, then some representation directions have extremely weak native signal scales, making them vulnerable to even localized parameter perturbations.

We formalize this intuition in four steps. First, we introduce the setup and the working assumptions under which the mechanism is analyzable (Section 4.1). Second, we define a relative-sensitivity metric and show why perturbations can be strongly amplified in low-singular-value directions (Section 4.2). Third, we discuss how such distortions accumulate under sequential editing and why practical behavior may become even worse once representation stability breaks down (Section 4.3). Finally, we bridge the representation-level analysis to model performance by exploring how excessive distortions correlate with downstream capability degradation, providing empirical evidence for the practical significance of this geometric vulnerability (Section 4.4).

Our goal is not to fully characterize edited LLM dynamics but to isolate a minimal vulnerability: dimensional collapse

makes representations fragile in low-variance directions, even in locally stable regimes.

4.1. Working Hypotheses and Problem Formulation

We consider a feed-forward network (FFN) layer in a Transformer model:

$$h = \phi(Wx), \quad (6)$$

where $x \in \mathbb{R}^{d_{in}}$ is the input vector, $W \in \mathbb{R}^{d \times d_{in}}$ is the weight matrix, $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is an elementwise nonlinearity, and $h \in \mathbb{R}^d$ is the resulting hidden representation.

A parameter-editing operation modifies the layer locally by adding a perturbation ΔW , producing $W_{edited} = W + \Delta W$.

Given a set of n input samples, let $H = [h^{(1)}, h^{(2)}, \dots, h^{(n)}] \in \mathbb{R}^{d \times n}$ denote the centered hidden representation matrix at this layer. We perform singular value decomposition: $H = U\Sigma V^T$.

where $U = [u_1, u_2, \dots, u_d] \in \mathbb{R}^{d \times d}$ is an orthonormal basis of principal directions, and the singular values are ordered as $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_d \geq 0$.

Assumption 4.1 (Dimensional Collapse Hypothesis). We assume that the hidden representation space of the pre-trained model exhibits dimensional collapse in a spectral sense: its singular-value spectrum is highly anisotropic, with a strongly compressed low-variance tail. Let $\sigma_{\min}$ be the smallest non-zero singular value. When $\sigma_{\min}$ is small, the associated direction has a weak signal scale.

This assumption is supported by empirical measurements, which show low effective rank, rapidly decaying singular values, and large condition numbers (e.g., Llama-3.1-8B-Instruct in Table 1). A detailed evaluation of these characteristics is provided in Appendix B.1 (Tables 5–8).

This setup allows us to ask a precise question: when an edit induces a change in hidden representations, how large is that change relative to the native signal scale of each principal direction?

Assumption 4.2 (Small Perturbation Hypothesis). A single editing operation induces a localized parameter perturbation ΔW whose norm is sufficiently small for a first-order Taylor approximation of the induced representation change to be informative.

For single-edit regimes, the induced hidden-state perturbation norms remain relatively small in many edited layers across several parameter-based editing methods, supporting the use of local first-order analysis as a descriptive approximation. Full experimental results validating this hypothesis are provided in Appendix B.2 (Tables 9–12).

Assumption 4.3 (Local Representation Stability Hypothesis). Within a short local editing regime, the principal directions u_k of the original representation space remain sufficiently stable to serve as an approximate common basis for analyzing perturbations. This should be interpreted as a local analytical idealization rather than a global claim, especially because low-variance directions may drift rapidly under sequential editing.

Empirical observations (Appendix B.3, Tables 13–20) suggest that dominant directions show higher stability under early sequential editing, while low-scale components deteriorate rapidly.

These hypotheses are not limited to a single model scale or family. A broader set of empirical measurements across different model sizes and architectures is provided in Appendix B.4, confirming the recurrence of the same spectral signatures and vulnerability mechanisms.

4.2. Relative Perturbation Amplification in Collapsed Directions

For a given input x , the representation change induced by editing is approximated by a first-order Taylor expansion (per Hypothesis 4.2):

$$\Delta h \triangleq h_{edited} - h = \phi((W + \Delta W)x) - \phi(Wx) \quad (7)$$

$$\approx J_\phi(a) \cdot \Delta W \cdot x, \quad (8)$$

where $a = Wx$ and $J_\phi(a) = \text{diag}(\phi'(a_1), \dots, \phi'(a_d)) \in \mathbb{R}^{d \times d}$ is the Jacobian matrix of activation function ϕ at a .

Under Hypothesis 4.3, we project Δh onto the original principal basis $\{u_k\}_{k=1}^d$:

$$\Delta h = \sum_{k=1}^d c_k u_k, \quad (9)$$

where $c_k = u_k^T \Delta h$ is the perturbation component along direction u_k .

To quantify the native signal scale of the unedited model in direction u_k , we consider the root mean square (RMS) magnitude across the sample set:

$$\text{RMS}(h_k) \triangleq \sqrt{\frac{1}{n} \sum_{i=1}^n (u_k^T h^{(i)})^2} = \frac{\sigma_k}{\sqrt{n}}. \quad (10)$$

For directions with non-zero σ_k , it determines the typical pre-edit scale of u_k .

Definition 4.4 (Directional Relative Change Rate). The relative change rate induced along direction u_k is defined as

$$R_k \triangleq \frac{c_k}{\text{RMS}(h_k)} = \frac{\sqrt{n}|c_k|}{\sigma_k}. \quad (11)$$

Table 1. Empirical evidence of dimensional collapse in the hidden representations of Llama-3.1-8B-Instruct. The hidden representation space exhibits clear characteristics of dimensional collapse: the effective rank r is significantly smaller than the feature dimension, the smallest singular value $\sigma_{\min}$ is extremely small ($\sim 10^{-5}$), and the condition number is very large ($> 10^6$).

layer	d	r_{eff}	r/d (%)	σ_1	$\sigma_{\min}$	$\text{cond}(\sigma_1/\sigma_{\min})$
5	4096	3249	79.3	194.1	3.38×10^{-6}	5.74×10^7
10	4096	3099	75.7	302.6	7.55×10^{-6}	4.01×10^7
15	4096	3177	77.6	504.9	1.20×10^{-4}	4.19×10^6
20	4096	3258	79.6	537.5	7.16×10^{-6}	7.50×10^7
25	4096	3260	79.6	618.7	9.48×10^{-6}	6.53×10^7
30	4096	2066	50.4	4922.3	3.85×10^{-5}	1.28×10^8
31	4096	1023	25.0	13003.3	3.05×10^{-3}	4.26×10^6

R_k quantifies the perturbation size relative to the native signal scale of direction u_k . A large R_k arises from either a large c_k or a small σ_k . When $R_k \ll 1$, the edit remains small compared to the typical pre-edit magnitude in that direction. In contrast, when $R_k \gtrsim 1$, the perturbation becomes comparable to or larger than the direction’s native signal scale, indicating substantial distortion.

Theorem 4.5 (Relative Amplification in Low-Scale Directions). *Let $u_{\min}$ denote a principal direction associated with the smallest non-zero singular value $\sigma_{\min}$. If the projection of the editing-induced perturbation onto this direction has magnitude $|c_{\min}| = \varepsilon$, then its directional relative change rate is*

$$R_{\min} = \frac{\sqrt{n} \varepsilon}{\sigma_{\min}}. \quad (12)$$

More generally, for any principal direction with $\sigma_k > 0$,

$$R_k = \frac{\sqrt{n} |c_k|}{\sigma_k}. \quad (13)$$

Hence, for perturbation components of comparable absolute magnitude across directions, the induced relative distortion is larger in directions with smaller singular values. In particular, when $\sigma_{\min}$ is extremely small, even a modest local perturbation can produce a very large relative distortion.

This theorem highlights a critical vulnerability: representation distortions are disproportionately amplified in directions with small σ_k irrespective of input perturbation energy. This concerns relative fragility, not semantic importance. We do not assume low-singular-value directions are meaningful or that editing concentrates perturbation energy in them. The claim is simply that any perturbation component landing there becomes disproportionately large relative to the native signal.

This amplification effect, where R_k is most pronounced along directions with small singular values, is empirically supported by Figure 2, which shows the distribution of R_k in layer 30 of Llama-3.1-8B-Instruct after performing 1000

sequential edits using MEMIT. Specifically, this observation aligns with our empirical measurements: the largest relative change rates concentrate in directions with small singular values, and a single edit can often yield $R_k > 1$ in many directions, exceeding the native signal scale of the original representation. Full empirical validations, including visualizations and statistics of R_k , are provided in Appendix B.5 (Figure 6, Tables 21–24).

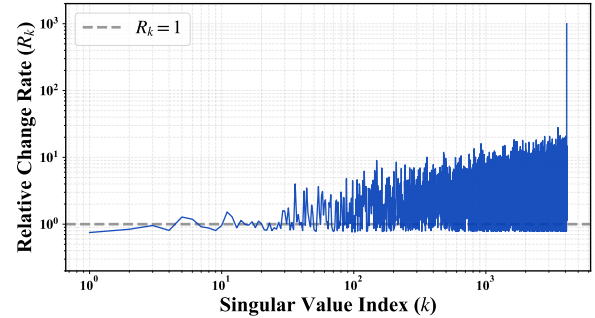


Figure 2. Distribution of R_k values in layer 30 of Llama-3.1-8B-Instruct after performing 1000 sequential edits using MEMIT.

4.3. Cumulative Effects in Sequential Editing

We now extend the analysis to sequential editing, where the model undergoes a series of updates: $W_{(0)} \rightarrow W_{(1)} \rightarrow \dots \rightarrow W_{(T)}$.

Let $h^{(t)}$ denote the hidden representation after the t -th edit. The total representation change after T edits relative to the original model satisfies the telescoping identity

$$\Delta h^{(T)} = h^{(T)} - h^{(0)} = \sum_{t=1}^T \Delta h_{\text{instant}}^{(t)}, \quad (14)$$

where $\Delta h_{\text{instant}}^{(t)} = h^{(t)} - h^{(t-1)}$ is the instantaneous change induced by the t -th edit.

Under the local stability approximation of Hypothesis 4.3,

Table 2. Spearman correlation between step-level amplification and editing performance, using the **median** of the layer-wise R_k distribution and **mean** aggregation across edited layers. Entries report Spearman’s ρ with significance markers, followed by the corresponding p -value in parentheses. *Note.* * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$.

Metric	AlphaEdit	MEMIT	ROME	WISE
Rel.	−0.2034 (0.5996)	−0.9619*** (< 0.001)	−0.8645** (0.0026)	−0.7679* (0.0157)
Gen.	0.6360 (0.0656)	−0.4238 (0.2557)	−0.8645** (0.0026)	−0.0840 (0.8298)
Loc.	0.0339 (0.9310)	−0.7289* (0.0259)	−0.9129*** (< 0.001)	−0.4851 (0.1857)
Port.	0.3500 (0.3558)	−0.8087** (0.0083)	−0.8617** (0.0028)	−0.6639 (0.0512)

we use the original principal basis as an approximate common coordinate system across edits. Projecting onto a principal direction $u_{\min}$ associated with the smallest non-zero singular value $\sigma_{\min}$, and defining $c_{\min}^{(t)} \triangleq u_{\min}^T \Delta h_{\text{inst}}^{(t)}$,

we obtain

$$c_{\min}^{(T)} = u_{\min}^T \Delta h^{(T)} = \sum_{t=1}^T c_{\min}^{(t)}. \quad (15)$$

In a worst-case scenario, where edits produce coherent accumulations of perturbations, we obtain:

$$|c_{\min}^{(T)}| \approx T \cdot \bar{\varepsilon}, \quad (16)$$

where $\bar{\varepsilon}$ is the average per-edit perturbation magnitude along $u_{\min}$. Substituting into Eq. (9) yields

$$R_{\min}^{(T)} = \frac{\sqrt{n} |c_{\min}^{(T)}|}{\sigma_{\min}} \approx T \cdot \frac{\sqrt{n} \bar{\varepsilon}}{\sigma_{\min}} = T \cdot R_{\min}^{(1)}. \quad (17)$$

Under this idealized coherent local regime, distortion in weak directions grows linearly with the number of edits.

This linear law assumes stable principal directions across edits, but in practice, this approximation breaks down as edits accumulate. Our empirical measurements show that principal directions, especially in the low-variance tail, drift rapidly under sequential editing across multiple models and architectures (Appendix B.3, Tables 13–20). Therefore, practical sequential editing does not merely accumulate perturbations in a fixed vulnerable basis; it can also reorganize the basis itself, inducing broader representational instability. This distinction strengthens the main message of the theory. The fixed-basis analysis shows that cumulative fragility already arises even in a favorable local regime. Once basis drift and representation shattering are taken into account, practical degradation can be even more severe than this idealized prediction suggests.

Taken together, Sections 4.2 and 4.3 suggest the following picture: dimensional collapse creates directions with extremely small native signal scales; small local edits are therefore disproportionately amplified in relative terms; and repeated editing compounds this vulnerability, eventually destabilizing the broader representation geometry.

4.4. From Representation Distortion to Capability Degradation

The analysis above is a representation-level statement. Large R_k does not directly imply behavioral failure but signifies structural risk. When R_k is large, the post-edit representation in u_k exceeds its native pre-edit scale.

To connect this to behavior, note that downstream computations depend not only on absolute activation magnitudes, but also on representation geometry, including relative positioning, subspace structure, and routing through subsequent nonlinear layers. Large relative distortions can therefore alter intermediate geometry in ways that propagate forward and potentially affect task behavior.

This interpretation is further supported by the correlation analysis between step-level R_k -based statistics and sequential editing performance. Table 2 reports the main Spearman correlation results, showing that larger amplification is generally associated with worse reliability, locality, and portability for several parameter-based editing methods. Full experimental details and complete results across summary and aggregation choices are provided in Appendix B.6. (Table 25–28).

5. Empirical Study

5.1. Experimental Settings

We evaluate on Llama-2-7B-Chat (Touvron et al., 2023), Llama-3.1-8B-Instruct (Meta AI, 2024), Mistral-7B-Instruct (Jiang et al., 2023), and DeepSeek-R1-Distill-LLaMA-8B (Guo et al., 2025), with larger-scale verification on Llama-2-13B and Qwen3-14B. The studied parameter-modifying methods span locate-then-edit (ROME (Meng et al., 2022b), MEMIT (Meng et al.), PMET (Li et al., 2024c), RECT (Gu et al., 2024b), AlphaEdit (Fang et al., 2025a), FT-L), meta-learning (MEND (De Cao et al., 2021)), and additional-parameter methods (AdaLoRA (Zhang et al., 2023a), WISE (Wang et al., 2024b)); external memory baselines include GRACE (Hartvigsen et al., 2024) and SCR (He et al., 2025). We use the ZsRE (Levy et al., 2017), WikiData_{counterfact} (Cohen et al., 2024), and ELKEN (Peng et al., 2024) datasets for editing, and mathematical reasoning datasets for general tasks. Evaluation adopts autoregressive

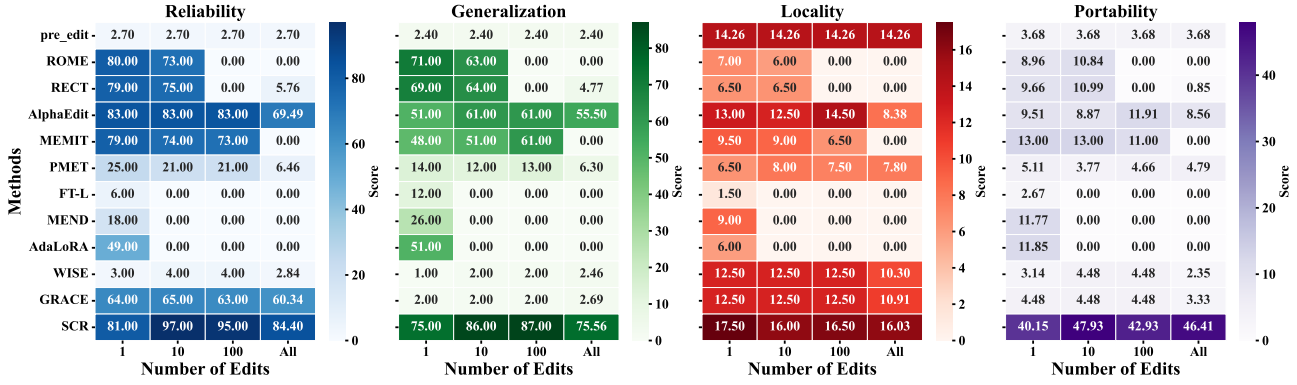


Figure 3. Performance changes of knowledge editing methods during sequential editing of Llama-3.1-8B-Instruct on the ZsRE dataset. The x-axis represents the number of edits: 1, 10, 100, and ‘All’ for the full dataset.

decoding with semantic consistency judged by Qwen2.5-72B-Instruct across four dimensions (Zhang et al., 2024b). A token-level locality check confirms the semantic results (Appendix C.5). Full implementation details, hardware specifications, and metric calculation procedures are provided in Appendix C.1.

5.2. Experimental Results

RQ1: How do knowledge editing methods perform in practice-oriented settings?

Figure 3 illustrates the intermediate performance of Llama-3.1-8B-Instruct on the ZsRE dataset as the number of edits increases from 1 to 10, 100, and finally to the full dataset. For the corresponding results of Llama-2-7B-Chat and Mistral-7B-Instruct, refer to Figure 7 and Figure 8 in the Appendix C.3, respectively. Appendix C.3 (Tables 29–31) presents a comprehensive comparison of knowledge editing methods across ZsRE, WikiData_{counterfact}, and larger LLMs such as Llama-2-13B and Qwen3-14B.

The experimental results reveal the following: (1) Under autoregressive inference and evaluation based on semantic consistency, all parameter modification-based methods fall significantly short of the near-perfect single-edit performance reported in prior work. (2) Most parameter modification-based methods, such as ROME and MEND, collapse under sequential editing scenarios: as the number of edits increases, all metrics quickly drop to near zero. This indicates a complete failure to retain knowledge across multiple updates. (3) AlphaEdit maintains a stable performance rate across continuous edits, its Reliability and Generalization scores on LLaMA-3.1-8B-Instruct are only 69.49 and 55.50, respectively. More importantly, it underperforms on Locality and Portability, with scores of just 8.38 and 8.58, highlighting its limited flexibility in applying edited knowledge. It is worth noting that AlphaEdit demonstrates sensitivity to hyperparameters and model architecture. Further analysis

can be found in Appendix C.6. (4) All parameter-based editing methods are outperformed by SCR, a simple and intuitive retrieval-based reference baseline without modifying model parameters. This highlights the practical potential of external-knowledge-based approaches in real-world knowledge updating tasks. Rather than serving as a direct competitor to parameter-based editing methods, SCR provides a distinct empirical perspective by operating outside the parametric editing paradigm, facilitating a principled study of the efficiency–reliability trade-off. Supplemental comparisons between SCR and parameter-based editing methods are reported in Appendix D (Tables 42 and 43).

A fine-grained analysis over a wider range of edit counts (Table 37 in Appendix C.3) further confirms that while the timing of degradation varies by method, all parameter-based editing methods exhibit performance decline over longer edit sequences. Additionally, there is consistent trends across both high- and low-confidence fact sets (Appendix C.3, Table 38).

RQ2: Can knowledge editing help reasoning LLMs integrate new facts without degrading reasoning ability?

As LLMs are increasingly applied to complex reasoning tasks, reasoning ability has become a key strength. Yet the effectiveness of knowledge editing methods for reasoning-oriented LLMs remains largely underexplored.

Tables 3 and 32 (Appendix C.3) present results on ZsRE dataset and WikiData_{counterfact}, respectively. To evaluate the retention of reasoning abilities after editing, we assess edited LLMs on a suite of mathematical reasoning benchmarks and broader downstream tasks (including GPQA-Diamond, ARC_c, and MMLU-Pro), reporting accuracy after 10 and 100 edits (Tables 33 and 34 in Appendix C.3). The results consistently show that reasoning performance collapses after extensive sequential edits for most parameter-based editing methods, while AlphaEdit exhibits relative resilience.

Table 3. Performance comparison of knowledge editing methods applied to DeepSeek-R1-Distill-Llama-8B on the ZsRE dataset, reporting results across four evaluation dimensions in both single and sequential editing settings, along with their average scores.

Method	Single Editing					Sequential Editing				
	Rel.	Gen.	Loc.	Port.	Avg.	Rel.	Gen.	Loc.	Port.	Avg.
<i>Pre-edit</i>	3.00	3.00	<u>15.50</u>	4.36	6.47	3.00	3.00	<u>15.50</u>	4.36	6.47
ROME	36.00	<u>42.00</u>	3.00	<u>17.99</u>	24.75	1.00	0.00	0.00	0.00	0.25
RECT	<u>37.00</u>	35.00	6.00	16.02	23.51	0.00	0.00	0.00	0.00	0.00
AlphaEdit	<u>43.00</u>	24.00	13.50	8.88	22.35	<u>46.00</u>	<u>35.00</u>	8.00	<u>7.62</u>	<u>24.16</u>
FT-L	2.00	2.00	2.00	3.93	2.48	0.00	0.00	0.00	0.00	0.00
MEND	36.00	<u>42.00</u>	10.50	15.47	<u>25.99</u>	0.00	0.00	0.00	0.00	0.00
AdaLoRA	18.00	15.00	0.50	8.03	10.38	0.00	0.00	0.00	0.00	0.00
WISE	8.00	5.00	3.00	2.59	4.65	2.00	2.00	7.50	2.52	3.50
GRACE	31.00	3.00	<u>15.50</u>	4.03	13.38	38.00	3.00	<u>15.50</u>	4.03	15.13
SCR	85.00	84.00	15.50	41.87	56.59	90.00	90.00	15.50	45.26	60.19

Based on the comparison between Table 3 and Table 29, knowledge editing methods that rely on parameter modification exhibit inferior performance on reasoning-oriented LLMs compared to general-purpose LLMs. For instance, AlphaEdit’s average performance in sequential editing scenarios drops from 35.48 to 24.16. This degradation may be due to the more implicit and distributed nature of knowledge representation in reasoning LLMs, as well as their greater dependence on long-range context reasoning. In contrast, SCR remains a strong practical reference baseline by integrating internal and external knowledge through selective context construction.

As shown in Table 33, ROME and RECT retain some performance, with average accuracy drops of 2.74 and 8.37, respectively. However, after 100 edits, their accuracy nearly drops to zero, indicating a complete collapse in reasoning ability. For MEND, accuracy remains at 0.00 after both 10 and 100 edits, suggesting that the method leads to total LLM failure. In contrast, AlphaEdit exhibits minimal performance degradation across all datasets, even after 100 sequential edits, and in some cases, exhibits slight improvements. Its parameter modifications can maintain reasoning accuracy close to that of the original LLM. Nevertheless, due to its limitations in locality and portability for knowledge editing, AlphaEdit may be better suited for general-purpose Parameter-Efficient Fine-Tuning rather than targeted knowledge editing tasks.

A detailed case analysis (see Appendix C.4) reveals that editing reasoning-oriented LLMs presents unique challenges. While editing methods can successfully guide the LLM to produce the correct next token, the edited LLM’s internal reasoning process often leads it to reflect outdated knowledge, thereby undermining the effectiveness of the edit. Moreover, in striving to preserve logical coherence within its reasoning trajectory, the edited LLM may generate explanations that are plausible-sounding but entirely fabricated. In some instances, the edited LLM may even disregard the orig-

inal question in its attempt to round out the answer. These behaviors underscore key limitations of current knowledge editing methods when applied to reasoning-oriented LLMs.

RQ3: Can knowledge editing methods generalize from factual knowledge to event knowledge?

While most existing research focuses on fact-based knowledge editing in the form of triples, real-world knowledge is often organized in a more complex manner, such as event.

However, current studies on knowledge editing rarely evaluate methods on more complex, event-level datasets. To address this limitation, we conduct knowledge editing experiments on Llama-2-7B-Chat, Llama-3.1-8B-Instruct, and DeepSeek-R1-Distill-Llama-8B using the ELKEN dataset (Peng et al., 2024), the dataset processing code using GPT-4o is fully described in Appendix C.2. Performance comparison between AlphaEdit and SCR on sequential editing tasks are shown in Figure 4. Detailed results for single editing and sequential editing are presented in Table 35 and Table 36 (Appendix C.3).

The results indicate that most parameter-based knowledge editing methods struggle to achieve satisfactory performance even in single-edit scenarios, and they almost completely break down under sequential editing. For instance, although AlphaEdit performs well on fact-based datasets, its performance deteriorates significantly when applied to event-level editing tasks involving multiple entities and attributes occurring concurrently. This degradation is primarily due to the limited ability of these methods to capture complex semantic relationships between entities and to integrate contextual information across interconnected elements via parameter-level adjustments. In contrast, SCR consistently achieves the best results across nearly all settings we evaluate. These findings underscore the practical limitations of parameter updates for editing small amounts of knowledge, while highlighting the substantial potential of in-context learning in LLMs for real-world applications.

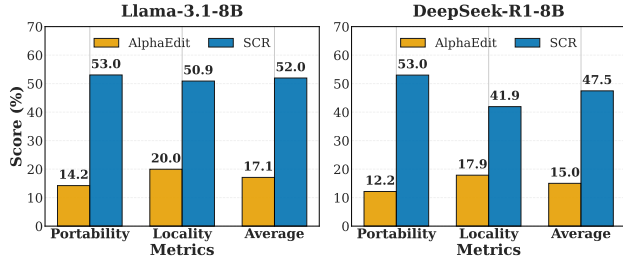


Figure 4. Performance comparison between AlphaEdit and SCR on sequential editing tasks using the ELKEN dataset.

RQ4: How do different knowledge editing methods compare in terms of time efficiency?

Beyond correctness and robustness, latency and efficiency are equally crucial for practical deployment. Effective editing methods should minimize both editing overhead and inference latency. To systematically evaluate these aspects, we consider two key metrics: (i) *edit time*, the wall-clock time required to apply an individual edit, and (ii) *inference time*, defined as the average wall-clock latency per input query normalized by the base LLM’s latency, measured under greedy decoding with a fixed output length of 50 tokens post-edit.

Figure 5 illustrates the trade-off between editing time and inference latency for parameter-based and external knowledge-based methods, with detailed results for all methods reported in Figure 9 in the Appendix C.3. Regarding editing time, methods that integrate knowledge through progressively more abstract representations, from textual edits to embeddings, hidden states, and ultimately parameter updates, exhibit increasing editing costs. This trend reflects the growing computational overhead associated with deeper and more indirect knowledge integration. In contrast, parameter-based editing methods do not introduce additional inference latency, as they directly modify the original model parameters. External knowledge-based methods, such as SCR, generally incur higher inference costs, primarily because of retrieval operations or the need to handle extended input sequences. Overall, no method achieves strong performance on both axes simultaneously. This reveals a fundamental limitation of current model editing paradigms: intrusive parameter updates trade update efficiency for inference efficiency, while non-intrusive methods preserve fast updates at the expense of increased inference latency. A more detailed analysis of SCR’s VRAM consumption, inference time, and performance stability across memory sizes and top-k settings is provided in Appendix E (Tables 44–46).

6. Conclusion

This paper shows that the limitations of parameter-based knowledge editing are rooted in the geometry of LLM repre-

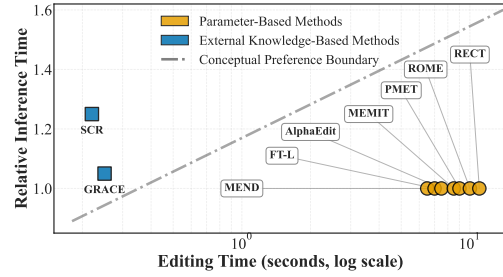


Figure 5. Trade-off between editing time and inference latency relative to the base model across different editing paradigms.

sentation spaces. Building on dimensional collapse, we develop a theoretical framework that models editing as a first-order perturbation of hidden representations and proves that bounded weight updates can induce disproportionately large distortions along low-variance principal directions, with such effects compounding under sequential editing. This identifies a concrete mechanism by which localized parameter modifications lead to global failures in reasoning and generalization. Extensive experiments under realistic settings corroborate these theoretical predictions: across varying knowledge complexity, numbers of edits, and evaluation dimensions, parameter-based editing methods consistently degrade core LLM capabilities, with performance deteriorating as edits accumulate, largely independent of the specific editing algorithm. Taken together, our results reveal a fundamental stability–efficiency trade-off in knowledge updating: methods that enable efficient, localized modifications can compromise the global stability of model representations, while mechanisms that better preserve representational integrity are often less efficient. Understanding, formalizing, and managing this trade-off is therefore a central challenge for future knowledge update methods.

Acknowledgements

This work was supported by the National Natural Science Foundation of China (Grant No. 72204087), the Chenguang Program of Shanghai Education Development Foundation and Shanghai Municipal Education Commission (Grant No. 23CGA28), the Shanghai Pujiang Program (Grant No. 23PJC030), and the Young Elite Scientists Sponsorship Program by CAST (Grant No. YESS20240562). We thank the ECNU LINK Lab for providing essential computational resources and a supportive research environment, which were instrumental in carrying out this study. We are also deeply grateful to the anonymous reviewers for their thoughtful comments and suggestions, which greatly improved the quality of this paper. Finally, we acknowledge the inspiring contributions of prior works in the field, as well as the open-source methods and datasets utilized in our study, which laid a critical foundation for this research.

Impact Statement

This paper establishes a rigorous, practice-oriented evaluation framework for knowledge editing methods, grounded in a geometric framework that explains why parameter-based knowledge editing methods may destabilize LLM representations. By revealing that even carefully targeted weight modifications systematically degrade core model capabilities, especially under realistic sequential editing conditions, our work encourages the community to shift focus from isolated factual accuracy metrics to adopting holistic evaluation standards that account for reliability, generalization, locality, portability, and reasoning preservation. We believe this shift in evaluation practice can steer future research toward more robust and trustworthy knowledge updating paradigms.

At the same time, we acknowledge two limitations of our analysis. First, our theoretical framework relies on specific assumptions (dimensional collapse, small perturbations, and local representation stability) and should be regarded as a falsifiable hypothesis rather than a conclusive theory. Second, although our findings highlight consistent shortcomings of current parameter-based editing methods, they should not be considered a wholesale rejection of parameter-based editing methods, which may still prove effective under carefully constrained conditions or with improved regularization. We hope this work inspires principled progress rather than deterring further exploration.

Finally, as with any technique enabling targeted modification of LLM behavior, advanced knowledge editing tools present inherent dual-use risks. However, the evaluation standards and diagnostic tools proposed in this work can help detect and mitigate potential misuse.

References

- AlKhamissi, B., Li, M., Celikyilmaz, A., Diab, M., and Ghazvininejad, M. A review on language models as knowledge bases. *arXiv preprint arXiv:2204.06031*, 2022.
- Allen-Zhu, Z. and Li, Y. Physics of language models: Part 3.3, knowledge capacity scaling laws. *arXiv preprint arXiv:2404.05405*, 2024.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.
- Chen, Q., Zhang, T., He, X., Li, D., Wang, C., Huang, L., et al. Lifelong knowledge editing for llms with retrieval-augmented continuous prompt learning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 13565–13580, 2024.
- Chen, Q., Wang, D., Zhang, T., Yan, Z., You, C., Wang, C., and He, X. Uneditit: A unified knowledge editing benchmark for large language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025. URL <https://openreview.net/forum?id=eESbQ5lWiP>.
- Chen, Z., Deng, Y., Wu, Y., Gu, Q., and Li, Y. Towards understanding the mixture-of-experts layer in deep learning. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, pp. 23049–23062, 2022.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457v1*, 2018.
- Cohen, R., Biran, E., Yoran, O., Globerson, A., and Geva, M. Evaluating the ripple effects of knowledge editing in language models. *Transactions of the Association for Computational Linguistics*, 12:283–298, 2024.
- Dai, D., Dong, L., Hao, Y., Sui, Z., Chang, B., and Wei, F. Knowledge neurons in pretrained transformers. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8493–8502, 2022.
- Dai, Y., Ji, Z., Li, Z., and Wang, S. Eamnet: Robust massive model editing via embedding alignment optimization. *arXiv preprint arXiv:2505.11876*, 2025.
- Das, P., Chaudhury, S., Nelson, E., Melnyk, I., Swaminathan, S., Dai, S., Lozano, A., Kollias, G., Chenthamarakshan, V., Navratil, J., et al. Larimar: Large language models with episodic memory control. In *Forty-first International Conference on Machine Learning*.
- De Cao, N., Aziz, W., and Titov, I. Editing factual knowledge in language models. In *EMNLP 2021-2021 Conference on Empirical Methods in Natural Language Processing, Proceedings*, pp. 6491–6506, 2021.
- Dong, Q., Dai, D., Song, Y., Xu, J., Sui, Z., and Li, L. Calibrating factual knowledge in pretrained language models. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pp. 5937–5947, 2022.
- Fang, J., Jiang, H., Wang, K., Ma, Y., Shi, J., Wang, X., He, X., and Chua, T.-S. Alphaedit: Null-space constrained model editing for language models. In *The Thirteenth International Conference on Learning Representations*, 2025a. URL <https://openreview.net/forum?id=HvSytvg3Jh>.

- Fang, J., Lu, T., Yao, Y., Jiang, Z., Xu, X., Chen, H., and Zhang, N. Cknowedit: A new chinese knowledge editing dataset for linguistics, facts, and logic error correction in llms. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8789–8807, 2025b.
- Geva, M., Bastings, J., Filippova, K., and Globerson, A. Dissecting recall of factual associations in auto-regressive language models. In *The 2023 Conference on Empirical Methods in Natural Language Processing*.
- Geva, M., Schuster, R., Berant, J., and Levy, O. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 5484–5495, 2021.
- Gu, J.-C., Xu, H.-X., Ma, J.-Y., Lu, P., Ling, Z.-H., Chang, K.-W., and Peng, N. Model editing harms general abilities of large language models: Regularization to the rescue. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 16801–16819, Miami, Florida, USA, November 2024a. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.934. URL <https://aclanthology.org/2024.emnlp-main.934/>.
- Gu, J.-C., Xu, H.-X., Ma, J.-Y., Lu, P., Ling, Z.-H., Chang, K.-W., and Peng, N. Model editing harms general abilities of large language models: Regularization to the rescue. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 16801–16819. Association for Computational Linguistics, 2024b.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Halevy, K., Sotnikova, A., Alkhamissi, B., Montariol, S., and Bosselut, A. “flex tape can’t fix that”: Bias and misinformation in edited language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 8690–8707, 2024.
- Hartvigsen, T., Sankaranarayanan, S., Palangi, H., Kim, Y., and Ghassemi, M. Aging with grace: Lifelong model editing with discrete key-value adaptors. *Advances in Neural Information Processing Systems*, 36, 2024.
- He, C., Luo, R., Bai, Y., Hu, S., Thai, Z., Shen, J., Hu, J., Han, X., Huang, Y., Zhang, Y., et al. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3828–3850, 2024.
- He, G., Song, X., and Sun, A. Knowledge updating? no more model editing! just selective contextual reasoning. *arXiv preprint arXiv:2503.05212*, 2025.
- Hu, C., Cao, P., Chen, Y., Liu, K., and Zhao, J. Wilke: Wise-layer knowledge editor for lifelong knowledge editing. In *Findings of the Association for Computational Linguistics ACL 2024*, pp. 3476–3503, 2024.
- Huang, Z., Shen, Y., Zhang, X., Zhou, J., Rong, W., and Xiong, Z. Transformer-patcher: One mistake worth one neuron. In *The Eleventh International Conference on Learning Representations*, 2023.
- Hurst, A., Lerer, A., Goucher, A. P., Perelman, A., Ramesh, A., Clark, A., Ostrow, A., Welihinda, A., Hayes, A., Radford, A., et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., Casas, D. d. l., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- Jiang, H., Fang, J., Zhang, N., Wan, M., Ma, G., Wang, X., He, X., and Chua, T.-S. Anyedit: Edit any knowledge encoded in language models. In *International Conference on Machine Learning*, pp. 27510–27533. PMLR, 2025.
- Jiang, Y., Wang, Y., Wu, C., Zhong, W., Zeng, X., Gao, J., Li, L., Jiang, X., Shang, L., Tang, R., et al. Learning to edit: Aligning llms with knowledge editing. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 4689–4705, 2024.
- Jiang, Z., Xu, F. F., Araki, J., and Neubig, G. How can we know what language models know? *Transactions of the Association for Computational Linguistics*, 8:423–438, 2020.
- Lee, S., Sim, W., Shin, D., Seo, W., Park, J., Lee, S., Hwang, S., Kim, S., and Kim, S. Reasoning abilities of large language models: In-depth analysis on the abstraction and reasoning corpus. *ACM Transactions on Intelligent Systems and Technology*.
- Levy, O., Seo, M., Choi, E., and Zettlemoyer, L. Zero-shot relation extraction via reading comprehension. In *21st Conference on Computational Natural Language Learning, CoNLL 2017*, pp. 333–342. Association for Computational Linguistics (ACL), 2017.

- Li, J., Beeching, E., Tunstall, L., Lipkin, B., Soletskyi, R., Huang, S., Rasul, K., Yu, L., Jiang, A. Q., Shen, Z., et al. Numinamath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions. *Hugging Face repository*, 13:9, 2024a.
- Li, Q. and Chu, X. Adaedit: Advancing continuous knowledge editing for large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 4127–4149, 2025.
- Li, S., Deng, Y., Cai, D., Lu, H., Chen, L., and Lam, W. Consecutive model editing with batch alongside hook layers. *arXiv preprint arXiv:2403.05330*, 2024b.
- Li, X., Li, S., Song, S., Yang, J., Ma, J., and Yu, J. Pmet: Precise model editing in a transformer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 18564–18572, 2024c.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step. In *International Conference on Learning Representations*, volume 2024, pp. 39578–39601, 2024.
- Liu, J., Sun, J., Shen, S., Yang, C., and Wang, S. Edit less, achieve more: Dynamic sparse neuron masking for lifelong knowledge editing in llms. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., and Neubig, G. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023.
- Lv, A., Zhang, K., Chen, Y., Wang, Y., Liu, L., Wen, J.-R., Xie, J., and Yan, R. Interpreting key mechanisms of factual recall in transformer-based language models. *arXiv preprint arXiv:2403.19521*, 2024.
- Markowitz, E., Ramakrishna, A., Mehrabi, N., Peris, C., Gupta, R., Chang, K.-W., and Galstyan, A. K-edit: Language model editing with contextual knowledge awareness. *arXiv preprint arXiv:2502.10626*, 2025.
- McCoy, R. T., Yao, S., Friedman, D., Hardy, M., and Griffiths, T. L. Embers of autoregression: Understanding large language models through the problem they are trained to solve. *arXiv preprint arXiv:2309.13638*, 2023.
- Meng, K., Sharma, A. S., Andonian, A. J., Belinkov, Y., and Bau, D. Mass-editing memory in a transformer. In *The Eleventh International Conference on Learning Representations*.
- Meng, K., Bau, D., Andonian, A., and Belinkov, Y. Locating and editing factual associations in gpt. *Advances in Neural Information Processing Systems*, 35:17359–17372, 2022a.
- Meng, K., Bau, D., Andonian, A., and Belinkov, Y. Locating and editing factual associations in gpt. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, pp. 17359–17372, 2022b.
- Meta AI. Llama 3 model card. https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md, 2024. Accessed: 2025-04-21.
- Mitchell, E., Lin, C., Bosselut, A., Finn, C., and Manning, C. D. Fast model editing at scale. *arXiv preprint arXiv:2110.11309*, 2021.
- Mitchell, E., Lin, C., Bosselut, A., Manning, C. D., and Finn, C. Memory-based model editing at scale. In *International Conference on Machine Learning*, pp. 15817–15831. PMLR, 2022.
- Mousavi, S. M., Alghisi, S., and Riccardi, G. Is your llm outdated? benchmarking llms & alignment algorithms for time-sensitive knowledge. *arXiv preprint arXiv:2404.08700*, 2024.
- Nie, E., Shao, B., Wang, M., Ding, Z., Schmid, H., and Schütze, H. Bmike-53: Investigating cross-lingual knowledge editing with in-context learning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16357–16374, 2025.
- Nishi, K., Ramesh, R., Okawa, M., Khona, M., Tanaka, H., and Lubana, E. S. Representation shattering in transformers: A synthetic study with knowledge editing. In Singh, A., Fazel, M., Hsu, D., Lacoste-Julien, S., Berkenkamp, F., Maharaj, T., Wagstaff, K., and Zhu, J. (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 46525–46553. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/nishi25a.html>.
- OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Park, H., Choi, G., Kim, M., and Jo, Y. Context-robust knowledge editing for language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 10360–10385, 2025.

- Peng, H., Wang, X., Li, C., Zeng, K., Duo, J., Cao, Y., Hou, L., and Li, J. Event-level knowledge editing. *arXiv preprint arXiv:2402.13093*, 2024.
- Petroni, F., Rocktäschel, T., Riedel, S., Lewis, P., Bakhtin, A., Wu, Y., and Miller, A. Language models as knowledge bases? In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 2463–2473, 2019.
- Pinter, Y. and Elhadad, M. Emptying the ocean with a spoon: Should we edit models? In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 15164–15172, 2023.
- Rawat, A. S., Zhu, C., Li, D., Yu, F., Zaheer, M., Kumar, S., and Bhojanapalli, S. Modifying memories in transformer models. In *International Conference on Machine Learning (ICML)*, volume 2020, 2021.
- Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., and Bowman, S. R. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*, 2023.
- Scialanga, M., Laugel, T., Grari, V., and Detryniecki, M. SAKE: Steering activations for knowledge editing. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15966–15978, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.777. URL <https://aclanthology.org/2025.acl-long.777/>.
- Sinitstin, A., Plokhhotnyuk, V., Pyrkin, D., Popov, S., and Babenko, A. Editable neural networks. *arXiv preprint arXiv:2004.00345*, 2020.
- Tan, C., Zhang, G., and Fu, J. Massive editing for large language models via meta learning. In *The Twelfth International Conference on Learning Representations*.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Valeriani, L., Doimo, D., Cuturello, F., Laio, A., Ansuini, A., and Cazzaniga, A. The geometry of hidden representations of large transformer models. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 51234–51252. Curran Associates, Inc., 2023.
- Wang, H., Liu, T., Li, R., Cheng, M., Zhao, T., and Gao, J. Roselora: Row and column-wise sparse low-rank adaptation of pre-trained language model for knowledge editing and fine-tuning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 996–1008, 2024a.
- Wang, J., Gu, Z., Zhu, X., Zhang, L., Ye, H., Xiong, Z., Jiang, S., Feng, H., and Xiao, Y. The missing piece in model editing: A deep dive into the hidden damage brought by model editing. In *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1–5. IEEE, 2025.
- Wang, P., Li, Z., Zhang, N., Xu, Z., Yao, Y., Jiang, Y., Xie, P., Huang, F., and Chen, H. WISE: Rethinking the knowledge memory for lifelong model editing of large language models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024b. URL <https://openreview.net/forum?id=VJMYOfJVC2>.
- Wang, R. and Li, P. Lemoe: Advanced mixture of experts adaptor for lifelong model editing of large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 2551–2575, 2024a.
- Wang, R. and Li, P. Memoe: Enhancing model editing with mixture of experts adaptors. *arXiv preprint arXiv:2405.19086*, 2024b.
- Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems*, 37:95266–95290, 2024c.
- Wei, Y., Yu, X., Weng, Y., Ma, H., Zhang, Y., Zhao, J., and Liu, K. Does knowledge localization hold true? surprising differences between entity and relation perspectives in language models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pp. 4118–4122, 2024.
- Williams, R. J. and Zipser, D. A learning algorithm for continually running fully recurrent neural networks. *Neural computation*, 1(2):270–280, 1989.
- Xie, J., Cao, P., Chen, Y., Liu, K., and Zhao, J. Revealing the deceptiveness of knowledge editing: A mechanistic analysis of superficial editing. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for*

- Computational Linguistics (Volume 1: Long Papers)*, pp. 17756–17780, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.868. URL <https://aclanthology.org/2025.acl-long.868/>.
- Yan, J., Wang, F., Luo, Y., Li, Y., and Zhang, Y. Keys to robust edits: from theoretical insights to practical advances. *arXiv preprint arXiv:2410.09338*, 2024.
- Yang, W., Sun, F., Ma, X., Liu, X., Yin, D., and Cheng, X. The butterfly effect of model editing: Few edits can trigger large language models collapse. In *Findings of the Association for Computational Linguistics ACL 2024*, pp. 5419–5437, 2024a.
- Yang, W., Sun, F., Tan, J., Ma, X., Su, D., Yin, D., and Shen, H. The fall of rome: Understanding the collapse of llms in model editing. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 4079–4087, 2024b.
- Yang, W., Sun, F., Tan, J., Ma, X., Cao, Q., Yin, D., Shen, H., and Cheng, X. The mirage of model editing: Revisiting evaluation in the wild. *arXiv preprint arXiv:2502.11177*, 2025.
- Yao, Y., Wang, P., Tian, B., Cheng, S., Li, Z., Deng, S., Chen, H., and Zhang, N. Editing large language models: Problems, methods, and opportunities. In *The 2023 Conference on Empirical Methods in Natural Language Processing*.
- Yu, L., Chen, Q., Zhou, J., and He, L. Melo: Enhancing model editing with neuron-indexed dynamic lora. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19449–19457, 2024.
- Zeng, A., Liu, X., Du, Z., Wang, Z., Lai, H., Ding, M., Yang, Z., Xu, Y., Zheng, W., Xia, X., Tam, W. L., Ma, Z., Xue, Y., Zhai, J., Chen, W., Liu, Z., Zhang, P., Dong, Y., and Tang, J. GLM-130b: An open bilingual pre-trained model. In *The Eleventh International Conference on Learning Representations*, 2023.
- Zhang, M., Ye, X., Liu, Q., Ren, P., Wu, S., and Chen, Z. Knowledge graph enhanced large language model editing. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 22647–22662, 2024a.
- Zhang, N., Yao, Y., Tian, B., Wang, P., Deng, S., Wang, M., Xi, Z., Mao, S., Zhang, J., Ni, Y., et al. A comprehensive study of knowledge editing for large language models. *arXiv preprint arXiv:2401.01286*, 2024b.
- Zhang, Q., Chen, M., Bukharin, A., Karampatziakis, N., He, P., Cheng, Y., Chen, W., and Zhao, T. Adalora: Adaptive budget allocation for parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.10512*, 2023a.
- Zhang, T., Chen, Q., Li, D., Wang, C., He, X., Huang, L., Huang, J., et al. Dafnet: Dynamic auxiliary fusion for sequential model editing in large language models. In *Findings of the Association for Computational Linguistics ACL 2024*, pp. 1588–1602, 2024c.
- Zhang, Y. and Math-AI, T. American invitational mathematics examination (aime) 2024, 2024.
- Zhang, Y. and Math-AI, T. American invitational mathematics examination (aime) 2025, 2025.
- Zhang, Z., Fang, M., Chen, L., Namazi-Rad, M.-R., and Wang, J. How do large language models capture the ever-changing world knowledge? a review of recent advances. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 8289–8311, 2023b.
- Zhang, Z., Geng, Z., Ma, Y., Wang, T., Lv, K., and Liu, X. Conflict-aware knowledge editing in the wild: Semantic-augmented graph representation for unstructured text. *Advances in Neural Information Processing Systems*, 38: 158954–158979, 2026.
- Zheng, C., Li, L., Dong, Q., Fan, Y., Wu, Z., Xu, J., and Chang, B. Can we edit factual knowledge by in-context learning? In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 4862–4876, 2023.
- Zhong, Z., Wu, Z., Manning, C. D., Potts, C., and Chen, D. Mquake: Assessing knowledge editing in language models via multi-hop questions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 15686–15702, 2023.

A. Related Work

Recent studies on updating factual knowledge in large language models have introduced diverse editing methods and evaluation protocols. In this section, we review representative knowledge editing approaches and summarize the experimental settings commonly used in prior work.

A.1. Knowledge Editing Methods

LLMs are often regarded as knowledge bases, as they encapsulate vast amounts of world knowledge within their extensive parameters, which are derived from large-scale datasets during the pre-training phase (Petroni et al., 2019; Geva et al., 2021; Geva et al.; Dai et al., 2022). To cope with knowledge updates, the knowledge editing approach (Meng et al., 2022b; Meng et al.) encodes target knowledge into specific parameters, which are then replaced or supplemented in the LLM to update its factual knowledge.

Locate-then-edit. methods (Meng et al.; Zhang et al., 2024a; Li et al., 2024c;b; Hu et al., 2024) are ground in the interpretability theory of Transformer architecture (Geva et al., 2021; Lv et al., 2024). It posits that knowledge is distributed across feed-forward networks (FFNs), while attention modules play a role in information copying and transmission. For instance, ROME (Meng et al., 2022b) utilizes causal tracing to first identify the crucial neurons associated with specific knowledge before performing targeted edits. Furthermore, RECT (Gu et al., 2024b) and AlphaEdit (Fang et al., 2025a) introduce additional constraints based on ROME to prevent excessive parameter shifts during the editing process. Note that, the assumption of the localized storage of factual knowledge remains controversial (Wei et al., 2024). An alternative hypothesis suggests that the relationship between neurons and knowledge is characterized by a many-to-many dynamic rather than a simplistic one-to-one association (Allen-Zhu & Li, 2024). Any modification to the parameters will inevitably affect other knowledge stored in the LLM, including both the original knowledge, and previously edited knowledge.

Meta-learning. methods, such as MEND (De Cao et al., 2021) and MALMEN (Tan et al.), employ hyper-networks that are designed to forecast tailored weight updates for each knowledge data instance associated with an LLM. However, the hyper-network for a particular LLM limits their scalability in sequential editing scenarios. Furthermore, the additional training process incurs significant time and computational costs. Additionally, the necessity of modifying parameters for a limited amount of knowledge encapsulated in textual form is a matter of ongoing debate.

Additional parameter-based. methods aim to efficiently integrate target knowledge by isolating the parameters that require adjustment, such as WISE (Wang et al., 2024b), T-Patcher (Huang et al., 2023), MELO (Yu et al., 2024) and many others (Dong et al., 2022; Zhang et al., 2024c; Wang & Li, 2024b;a; Wang et al., 2024a). These methods typically introduce additional parameters or utilize mixture of experts (MoE) architectures (Chen et al., 2022), either at the head of the LLM or within its structure. However, as the number of additional parameters increases, the likelihood of overfitting escalates. This phenomenon can result in the post-edit LLM neglecting prior edits, and compromising its original knowledge. Besides, the continual expansion of neurons may further exacerbate the post-edit LLM’s inference burden.

External Memory-based. methods (Zhong et al., 2023; Hartvigsen et al., 2024; Mitchell et al., 2022; Jiang et al., 2024; Chen et al., 2024; Das et al.; Zheng et al., 2023; Markowitz et al., 2025) maintain a memory store for updated knowledge, which can be represented as plain text, hidden states, token embeddings, or knowledge graphs. SERAC (Mitchell et al., 2022), a classical method, simulates the editing scope by training a discriminator, whose results distinguish between the original LLM and the counterfactual model. GRACE (Hartvigsen et al., 2024) maintains a dynamically updated codebook that alters the hidden states during the forward propagation.

In short, most existing methods update LLM knowledge by modifying their parameters or structures. However, in real-world scenarios, the amount of new knowledge available is limited, compared to the vast pre-training data used by LLMs. Encoding new knowledge into the model’s parameters can lead to the loss of original knowledge, risking both an incomplete understanding of updates and potential conflicts with prior knowledge.

A.2. Summary of Experimental Settings

A comprehensive summary of the experimental settings adopted by recent methods is provided in Table 4.

Table 4. Summary of experimental settings for recent methods. We specify whether **Fact**-, **Event**-based, or **General** purpose **Datasets** are used. Target **LLM** specifies whether an **Instruct** LLM or an **Reasoning** LLM is employed. **Inference** settings show whether **Autoregressive** setting is applied instead of teacher-forcing. Under **Edits**, we clarify if **Single** or **Sequential** editing is tested. For evaluation **Dimensions**, we specify if **Reliability**, **Generalization**, **Locality**, and **Portability** are considered.

Category	Method	Fact	Datasets		Instruct	Reason	Infer Auto	Edits		Rel.	Dimensions		
			Event	Gene.				Single	Seq.		Gen.	Loc.	Port.
Locate-then-edit	ROME (Meng et al., 2022b)	✓	✗	✗	✓	✗	✗	✓	✗	✓	✓	✓	✗
	MEMIT (Meng et al.)	✓	✗	✗	✓	✗	✗	✓	✓	✓	✓	✓	✗
	PMET (Li et al., 2024c)	✓	✗	✗	✓	✗	✗	✓	✓	✓	✓	✓	✗
	RECT (Gu et al., 2024b)	✓	✗	✗	✓	✗	✗	✓	✓	✓	✓	✓	✗
	AlphaEdit (Fang et al., 2025a)	✓	✗	✓	✓	✗	✗	✓	✓	✓	✓	✓	✓
	FT-L (Meng et al., 2022b)	✓	✗	✗	✓	✗	✗	✓	✗	✓	✓	✓	✗
	CoRE (Park et al., 2025)	✓	✗	✓	✓	✗	✓	✓	✗	✓	✓	✓	✗
	EAMET (Dai et al., 2025)	✓	✗	✓	✓	✗	✗	✓	✓	✓	✓	✓	✓
	AnyEdit (Jiang et al., 2025)	✓	✓	✗	✓	✗	✓	✓	✗	✓	✓	✓	✗
Meta-learning	MEND (De Cao et al., 2021)	✓	✗	✗	✓	✗	✗	✓	✗	✓	✓	✓	✗
Additional-Parameter	AdaLoRA (Zhang et al., 2023a)	✓	✗	✗	✓	✗	✗	✓	✗	✓	✓	✓	✗
	WISE (Wang et al., 2024b)	✓	✗	✗	✓	✗	✗	✓	✓	✓	✓	✓	✗
	AdaEdit (Li & Chu, 2025)	✓	✗	✓	✓	✗	✗	✓	✓	✓	✓	✓	✗
External Memory	GRACE (Hartvigsen et al., 2024)	✓	✗	✗	✓	✗	✗	✓	✓	✓	✓	✓	✗

B. Justifications for Working Hypotheses and Empirical Support

B.1. Hypothesis 4.1: Dimensional Collapse

This subsection evaluates whether the hidden representations in our studied models exhibit the spectral characteristics required by Hypothesis 4.1.

Literature Motivation

Prior research on the geometry of representations in neural networks has consistently documented phenomena such as anisotropy, rank collapse, and low-dimensional concentration in Transformer hidden states. These findings suggest that hidden representation matrices can be effectively modeled as having a compressed singular value spectrum. Recent empirical studies (Valeriani et al., 2023) specifically indicate that hidden representations in intermediate layers of large Transformer models experience a significant contraction in intrinsic dimensionality. This contraction implies that the representations lie near a low-dimensional manifold with a small effective rank and a rapidly decaying singular value spectrum, supporting the assumption of a near-zero minimum singular value ($\sigma_{\min}$) in our hypothesis.

Further supporting evidence comes from Pinter & Elhadad (2023), who observe that editing operations in large language models (LLMs) cause a pervasive "ripple effect" in the hidden representation space. This suggests structural fragility, which aligns with the geometric characteristics of dimensional collapse. Additionally, mechanistic analyses (Xie et al., 2025) link specific editing failures to low-variance substructures in attention modules, highlighting the vulnerability of critical information encoded in directions with low variance and sensitivity to perturbation.

Hidden State Extraction

For each evaluated model (Llama-3.1-8B-Instruct, DeepSeek-R1-Distill-Llama-8B, Llama-2-13B), we sample $n = 100000$ input prompts from the C4-en dataset. Hidden states $h \in \mathbb{R}^d$ are then extracted using forward hooks implemented in PyTorch during forward passes, capturing layer-specific hidden representations before and after each editing operation. Specifically, hidden states are collected from the down-projection layer of each Transformer block’s feed-forward network. For the T5-3B encoder-decoder model, hidden representations are captured at the input and output projections of feed-forward networks in the layers targeted by MEND (i.e., encoder layers 22–23, decoder layers 22–23), following the default hyperparameter settings for MEND.

Singular Value Spectrum Analysis

For each layer, we construct the hidden representation matrix $H \in \mathbb{R}^{d \times n}$, where d represents the feature dimension and $n = 100000$ is the number of samples. A full singular value decomposition (SVD) $H = U\Sigma V^\top$ is performed using PyTorch’s GPU-accelerated `torch.linalg.svd` implementation. This yields the ordered singular value spectrum $\{\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_d\}$, which provides insights into the representation space’s intrinsic dimensionality and capacity.

Empirical Validation

We empirically analyze the singular value spectra of hidden representations across models and layers:

- Table 5 reports dimensional collapse statistics for Llama-3.1-8B-Instruct.
- Table 6 reports the same analysis for DeepSeek-R1-Distill-LLaMA-8B.
- Table 7 reports the same analysis for LLaMA-2-13B.
- Table 8 reports the same analysis for selected encoder and decoder projections in T5-3B.

Across all models, architectures, and configurations, we consistently observe the following trends: (1) low effective rank, (2) vanishingly small singular values in the spectral tail, and (3) large condition numbers. Although the degree of collapse varies by layer and architecture, these results strongly align with the hypothesis of dimensional collapse. Consequently, they provide empirical support for interpreting dimensional collapse as the spectral precondition underlying the relative amplification mechanisms explored in Section 4.2.

Table 5. Empirical evidence of dimensional collapse in the hidden representations of Llama-3.1-8B-Instruct.

Layer	d	r_{eff}	r/d (%)	σ_1	$\sigma_{\min}$	$\text{cond}(\sigma_1/\sigma_{\min})$
5	4096	3249	79.3	1.9×10^2	3.4×10^{-6}	5.7×10^7
10	4096	3099	75.7	3.0×10^2	7.5×10^{-6}	4.0×10^7
15	4096	3177	77.6	5.0×10^2	1.2×10^{-4}	4.2×10^6
20	4096	3258	79.6	5.4×10^2	7.2×10^{-6}	7.5×10^7
25	4096	3260	79.6	6.2×10^2	9.5×10^{-6}	6.5×10^7
30	4096	2066	50.4	4.9×10^3	3.9×10^{-5}	1.3×10^8
31	4096	1023	25.0	1.3×10^4	3.0×10^{-3}	4.3×10^6

Table 6. Empirical evidence of dimensional collapse in the hidden representations of DeepSeek-R1-Distill-Llama-8B.

Layer	d	r_{eff}	r/d (%)	σ_1	$\sigma_{\min}$	$\text{cond}(\sigma_1/\sigma_{\min})$
5	4096	3269	79.8	2.2×10^2	3.2×10^{-6}	6.7×10^7
10	4096	3087	75.4	3.6×10^2	4.8×10^{-6}	7.5×10^7
15	4096	3168	77.4	5.9×10^2	7.0×10^{-6}	8.5×10^7
20	4096	3371	82.3	4.3×10^2	7.1×10^{-6}	6.1×10^7
25	4096	3403	83.1	5.6×10^2	8.5×10^{-6}	6.6×10^7
30	4096	2450	59.8	3.1×10^3	3.1×10^{-5}	1.0×10^8
31	4096	1395	34.1	7.1×10^3	6.5×10^{-5}	1.1×10^8

Table 7. Empirical evidence of dimensional collapse in the hidden representations of Llama-2-13B.

Layer	d	r_{eff}	r/d (%)	σ_1	$\sigma_{\min}$	$\text{cond}(\sigma_1/\sigma_{\min})$
5	5120	3941	77.0	2.8×10^2	5.2×10^{-6}	5.5×10^7
10	5120	3956	77.3	4.0×10^2	1.0×10^{-5}	4.0×10^7
15	5120	4089	79.9	5.6×10^2	1.4×10^{-5}	4.0×10^7
20	5120	4177	81.6	1.0×10^3	1.9×10^{-5}	5.3×10^7
25	5120	4249	83.0	9.6×10^2	2.3×10^{-5}	4.1×10^7
30	5120	4224	82.5	1.6×10^3	2.4×10^{-5}	6.5×10^7
31	5120	4238	82.8	1.4×10^3	2.3×10^{-5}	6.0×10^7
35	5120	4087	79.8	2.1×10^3	2.8×10^{-5}	7.5×10^7
39	5120	1448	28.3	2.6×10^4	1.8×10^{-4}	1.5×10^8

Table 8. Empirical evidence of dimensional collapse in T5-3B hidden representations at MEND editing layers.

Layer	d	r_{eff}	r/d (%)	σ_1	$\sigma_{\min}$	$\text{cond}(\sigma_1/\sigma_{\min})$
Enc22-FFN-wi	16384	839	5.1	1.9×10^4	3.3×10^{-9}	5.7×10^{12}
Enc22-FFN-wo	1024	31	3.1	1.7×10^6	6.0×10^1	2.8×10^4
Enc23-FFN-wi	16384	1194	7.3	1.5×10^4	4.0×10^{-9}	3.6×10^{12}
Enc23-FFN-wo	1024	129	12.6	6.8×10^5	5.6×10^1	1.2×10^4
Dec22-FFN-wi	16384	3334	20.3	6.5×10^3	2.6×10^{-9}	2.5×10^{12}
Dec22-FFN-wo	1024	447	43.6	1.1×10^5	1.4×10^2	8.3×10^2
Dec23-FFN-wi	16384	5997	36.6	1.5×10^5	2.9×10^{-6}	5.1×10^{10}
Dec23-FFN-wo	1024	94	9.2	8.3×10^6	1.2×10^3	6.7×10^3

B.2. Hypothesis 4.2: Small-Perturbation Regime

This subsection examines whether single-edit perturbations remain sufficiently small to render the first-order approximation in Eq. (8) an informative analytical framework.

Literature Motivation

Empirical research on the unintended side effects of editing models highlights a strong connection between the magnitude of weight updates and the degradation of model performance. Large weight updates, characterized by high Frobenius norms $\|\Delta W\|_F$, have been shown to induce overfitting and diminish generalization capabilities (Gu et al., 2024a). In contrast, editing techniques such as RECT, which explicitly constrain the magnitude of weight changes, have demonstrated effectiveness in mitigating these adverse effects (Gu et al., 2024a). This body of evidence underscores the importance of small, localized parameter perturbations for successful editing. Further validation comes from failure analyses, which link catastrophic model collapse directly to the generation of excessively large parameter updates (Yang et al., 2024b). Collectively, these findings justify the adoption of a first-order Taylor expansion as a locally linear approximation to analyze the effects of small edits.

Perturbation Norm Measurement

We evaluate five editing methods (AlphaEdit, MEMIT, MEND, ROME, WISE) by applying a single edit to each model and measuring the resulting hidden state changes across layers. Specifically, we compute the hidden-state perturbation $\Delta h = h_{\text{edit}} - h_{\text{orig}}$ for each edited layer and record its ℓ_2 norm $\|\Delta h\|$ over $n = 100000$ input samples. To test Assumption 4.2, which posits that single-edit perturbations remain in the small regime where a first-order Taylor approximation is appropriate, we report the mean, standard deviation, minimum, and maximum of these norms.

Empirical Validation

We quantify the norm of hidden-state perturbations induced by single edits across a variety of methods and models:

- Table 9 reports single-edit perturbation norms in Llama-3.1-8B-Instruct.
- Table 10 reports the same analysis for DeepSeek-R1-Distill-LLaMA-8B.
- Table 11 reports the same analysis for LLaMA-2-13B.
- Table 12 reports the same analysis for T5-3B under MEND.

In decoder-only models, perturbation norms are generally small in most edited layers, supporting the validity of local first-order analyses for single edits. However, the results for T5 reveal a greater degree of layerwise heterogeneity, suggesting that the small-perturbation regime is influenced by both architectural and layer-specific factors. These findings support the use of Eq. (8) as a local analytical approximation for understanding single-edit effects, while also clarifying its limitations in more heterogeneous settings.

Table 9. Perturbation norms induced by a single edit in selected layers of Llama-3.1-8B-Instruct.

Method	Layer	mean($\ \Delta h\ $)	std	min	max
AlphaEdit	5	1.24	0.12	0.00	2.18
	10	1.79	0.14	0.01	3.92
	15	2.64	0.28	0.01	5.12
	20	2.71	0.33	0.01	6.01
	25	3.27	0.48	0.01	8.05
	30	9.00	1.66	0.02	23.10
	31	0.05	0.03	0.01	1.17
MEMIT	5	1.24	0.12	0.00	2.18
	10	1.79	0.14	0.00	3.92
	15	2.64	0.28	0.00	5.12
	20	2.71	0.33	0.00	6.01
	25	3.27	0.48	0.00	8.05
	30	9.00	1.66	0.01	23.10
	31	0.03	0.01	0.01	0.53
MEND	29	0.46	0.16	0.08	2.11
	30	0.36	0.16	0.07	3.06
	31	0.75	0.31	0.19	5.89
ROME	5	1.28	0.12	0.27	2.21
	10	1.79	0.14	0.10	3.92
	15	2.64	0.28	0.11	5.12
	20	2.71	0.33	0.09	6.04
	25	3.27	0.48	0.12	8.11
	30	9.00	1.66	0.38	23.10
	31	0.53	0.27	0.12	13.76
WISE	29	0.13	1.71	0.01	45.38
	30	0.07	0.77	0.01	19.99
	31	0.20	2.28	0.02	70.90

Table 10. Perturbation norms induced by a single edit in selected layers of DeepSeek-R1-Distill-Llama-8B.

Method	Layer	mean($\ \Delta h\ $)	std	min	max
AlphaEdit	5	1.15	0.11	0.00	2.06
	10	1.67	0.13	0.02	2.75
	15	2.44	0.24	0.01	4.23
	20	2.58	0.29	0.01	5.76
	25	3.22	0.43	0.02	6.88
	30	7.19	1.01	0.04	15.88
	31	0.08	0.03	0.02	0.80
MEMIT	5	1.15	0.11	0.00	2.06
	10	1.67	0.13	0.01	2.76
	15	2.44	0.24	0.01	4.23
	20	2.58	0.29	0.01	5.76
	25	3.22	0.43	0.01	6.88
	30	7.20	1.01	0.02	15.89
	31	0.03	0.01	0.01	0.22
MEND	29	0.46	0.20	0.03	1.91
	30	7.19	1.00	0.26	15.81
	31	0.70	0.27	0.11	2.60
ROME	5	1.19	0.11	0.31	2.07
	10	1.67	0.13	0.11	2.80
	15	2.44	0.24	0.10	4.25
	20	2.58	0.29	0.09	5.76
	25	3.22	0.44	0.14	6.88
	30	7.20	1.01	0.30	15.97
	31	0.42	0.16	0.13	4.59
WISE	29	15.28	1.79	0.00	24.00
	30	8.11	1.01	3.69	15.94
	31	11.32	1.31	0.01	18.73

Table 11. Perturbation norms induced by a single edit in selected layers of Llama-2-13B.

Method	Layer	mean($\ \Delta h\ $)	std	min	max
AlphaEdit	5	2.07	0.21	1.17	3.69
	10	4.19	0.37	2.78	7.54
	15	6.06	0.55	3.65	10.97
	20	8.04	0.96	4.69	20.12
	25	9.83	1.45	4.95	21.32
	30	9.36	1.32	5.13	25.31
	35	0.02	0.02	0.01	0.60
	39	0.07	0.05	0.02	2.69
MEMIT	5	2.07	0.21	1.17	3.70
	10	4.19	0.36	2.78	7.54
	15	6.06	0.55	3.65	10.90
	20	8.04	0.96	4.69	20.12
	25	9.83	1.45	4.95	21.37
	30	9.36	1.32	5.15	25.29
	35	0.06	0.04	0.02	1.23
	39	0.17	0.18	0.03	11.38
MEND	37	0.25	0.02	0.17	0.54
	38	0.38	0.04	0.20	0.76
	39	1.43	0.17	0.58	6.15
ROME	5	2.09	0.21	1.18	3.71
	10	4.19	0.36	2.78	7.55
	15	6.06	0.55	3.65	10.98
	20	8.04	0.96	4.69	20.12
	25	9.83	1.45	4.95	21.34
	30	9.37	1.32	5.12	25.37
	35	0.12	0.09	0.02	3.23
	39	0.32	0.30	0.06	29.98
WISE	27	0.05	0.01	0.03	0.66
	30	9.37	1.32	5.13	25.33
	35	0.07	0.02	0.04	0.89
	39	0.91	0.26	0.18	5.43

Table 12. Perturbation norms induced by a single MEND edit in selected layers of T5-3B.

Layer	mean($\ \Delta h\ $)	std	min	max
Enc22-FFN-wi	0.16	0.01	0.11	0.28
Enc22-FFN-wo	42.21	9.94	10.97	134.68
Enc23-FFN-wi	0.47	0.04	0.31	0.71
Enc23-FFN-wo	822.34	107.67	359.91	1.57×10^3
Dec22-FFN-wi	21.68	2.58	14.05	49.77
Dec22-FFN-wo	125.57	24.62	63.19	394.57
Dec23-FFN-wi	133.78	68.82	37.00	736.76
Dec23-FFN-wo	3.76×10^3	3.77×10^3	865.09	4.89×10^4

B.3. Hypothesis 4.3: Local Representation Stability

This subsection investigates whether principal directions in the representation space remain approximately stable under sequential editing.

Literature Motivation

Recent studies have shown that model editing can substantially distort the global geometry of hidden representations, a phenomenon termed "representation shattering," which is strongly correlated with performance degradation (Nishi et al., 2025). These findings suggest that maintaining the structural stability of the representation space is essential for preserving model functionality after editing. Furthermore, widespread observations of unintended geometric disruptions, such as the hidden space ripple effect (Wang et al., 2025), reinforce concerns about the challenges of achieving stable, localized updates. This aligns with broader skeptical perspectives on the feasibility of treating large language models (LLMs) as reliably editable knowledge repositories (Pinter & Elhadad, 2023). We therefore adopt the approximate stability of principal component directions u_k as a working hypothesis to describe a stable editing regime.

Principal Component Stability Analysis

In the context of sequential editing ($T = 1, 2, \dots, \text{all edits}$), we extract the principal component matrices $U_{\text{edit}}^{(T)}$ from the hidden states after each T -th edit and compare them to the original principal component matrix U_{orig} of the unedited model. To assess the geometric consistency between the top- k principal components of $U_{\text{edit}}^{(T)}$ and U_{orig} , we calculate the minimum principal angle $\theta_{\min}$ between their respective subspaces. To further quantify alignment, we compute the cosine of the minimum angle, $\cos \theta_{\min}$. A value of $\cos \theta_{\min}$ close to 1 indicates strong alignment and stability, while values approaching 0 suggest near-orthogonality and instability. This analysis is performed across a range of subspace dimensions $k \in \{10, 50, 100, 200, 500, 1000, 2000\}$.

Empirical Validation

We evaluate the cosine similarity between the original principal directions and those obtained after sequential edits to quantify the stability of principal component directions under sequential editing. We perform this analysis for both decoder-only and encoder-decoder models. For decoder-only models, including Llama-3.1-8B-Instruct, DeepSeek-R1-Distill-LLaMA-8B, and LLaMA-2-13B, we conduct experiments with AlphaEdit and ROME. For the encoder-decoder model T5-3B, we examine the FFN input-projection (W_i) and output-projection (W_o) spaces under edits performed with MEND. The results are summarized in the following tables:

- Tables 13, 14, and 15 report the cosine similarity between the original and edited principal directions at layer 30 of Llama-3.1-8B-Instruct, DeepSeek-R1-Distill-LLaMA-8B, and LLaMA-2-13B, respectively, under sequential edits with AlphaEdit.
- Tables 16, 17, and 18 report the corresponding results for the same models under sequential edits with ROME.
- Tables 19 and 20 report the cosine similarity between the original and edited principal directions in the FFN input-projection and output-projection spaces of T5-3B under sequential edits with MEND.

Across models, dominant PCs often remain relatively stable during the initial edits, while lower-variance PCs deteriorate rapidly as edits accumulate. In T5-3B, certain layers exhibit a loss of stability almost immediately after editing begins. These observations reinforce the interpretation of Hypothesis 4.3 as a short-horizon idealization and underscore that the practical degradation of representation geometry may exceed the predictions of this simplified scenario.

B.4. Generalization Across Model Scales and Architectures

The main text focuses on 7B–14B decoder-only LLMs, but the geometric signatures underlying our analysis extend beyond a single model. The dimensional-collapse statistics (Tables 5–8), perturbation norms (Tables 9–12), PC-stability analyses (Tables 13–20), and R_k distributions (Tables 21–24) consistently reveal evidence of the same vulnerability mechanism across DeepSeek-R1-Distill-LLaMA-8B, LLaMA-2-13B, and specific components of T5-3B.

However, the encoder-decoder architecture exhibits greater heterogeneity, with certain layers departing from the small-perturbation regime more rapidly. As a result, we do not assert identical quantitative patterns across all architectures. Instead,

Table 13. Stability of principal component directions at layer 30 of Llama-3.1-8B-Instruct under sequential editing using AlphaEdit.

Edit Step	Top10 PCs Cos	Top100 PCs Cos	Top200 PCs Cos	Top2000 PCs Cos
1	0.999	0.981	0.927	0.003
2	1.000	1.000	1.000	0.940
3	1.000	1.000	1.000	0.936
4	0.999	0.981	0.927	0.002
5	1.000	1.000	0.999	0.872
6	0.999	0.981	0.927	0.003
7	1.000	1.000	0.998	0.917
8	0.999	0.981	0.926	0.003
9	1.000	0.999	0.997	0.866
10	0.999	0.981	0.926	0.004
20	0.999	0.981	0.925	0.006
50	0.997	0.993	0.956	0.299
100	0.996	0.980	0.910	0.002
200	0.994	0.966	0.802	0.002
500	0.990	0.813	0.379	0.003
1000	0.973	0.552	0.128	0.002
all	0.962	0.447	0.067	0.020

Table 14. Stability of principal component directions at layer 30 of DeepSeek-R1-Distill-Llama-8B under sequential editing using AlphaEdit.

Edit Step	Top10 PCs Cos	Top100 PCs Cos	Top200 PCs Cos	Top2000 PCs Cos
1	1.000	0.984	0.906	0.014
2	0.999	0.984	0.902	0.015
3	0.999	0.984	0.902	0.014
4	0.999	0.984	0.904	0.015
5	0.999	0.984	0.897	0.014
6	0.999	0.984	0.898	0.014
7	1.000	0.978	0.996	0.878
8	0.999	0.975	0.901	0.015
9	0.999	0.961	0.908	0.016
10	0.999	0.920	0.901	0.016
20	0.999	0.840	0.871	0.011
50	0.997	0.910	0.751	0.005
100	0.994	0.446	0.207	0.105
200	0.989	0.086	0.207	0.053
500	0.979	0.169	0.283	0.033
1000	0.958	0.199	0.021	0.018
all	0.940	0.139	0.015	0.000

Table 15. Stability of principal component directions at layer 30 of Llama-2-13B under sequential editing using AlphaEdit.

Edit Step	Top10 PCs Cos	Top100 PCs Cos	Top200 PCs Cos	Top2000 PCs Cos
1	0.999	0.981	0.946	0.040
2	0.999	0.981	0.946	0.039
3	0.999	0.981	0.946	0.039
4	0.999	0.980	0.946	0.040
5	0.999	0.981	0.946	0.040
6	0.999	0.980	0.946	0.039
7	0.999	0.980	0.946	0.040
8	0.999	0.980	0.946	0.039
9	0.999	0.980	0.946	0.039
10	0.999	0.980	0.946	0.038
20	0.999	0.978	0.946	0.038
50	0.999	0.971	0.942	0.029
100	0.999	0.929	0.932	0.528
200	0.998	0.904	0.372	0.039
500	0.997	0.159	0.268	0.001
1000	0.986	0.230	0.441	0.005
all	0.972	0.101	0.313	0.013

Table 16. Stability of principal component directions at layer 30 of Llama-3.1-8B-Instruct under sequential editing using ROME.

Edit Step	Top10 PCs Cos	Top100 PCs Cos	Top200 PCs Cos	Top2000 PCs Cos
1	0.999	0.981	0.926	0.002
2	0.997	0.995	0.974	0.517
3	0.995	0.989	0.908	0.441
4	0.983	0.974	0.825	0.001
5	0.979	0.977	0.764	0.116
6	0.979	0.970	0.619	0.002
7	0.980	0.861	0.585	0.001
8	0.964	0.928	0.388	0.001
9	0.971	0.881	0.413	0.039
10	0.971	0.893	0.412	0.001
20	0.274	0.004	0.003	0.001
50	0.029	0.004	0.002	0.000
100	0.005	0.000	0.000	0.000
200	0.001	0.001	0.000	0.000
500	0.004	0.001	0.000	0.000
1000	0.012	0.000	0.001	0.000
all	0.012	0.001	0.002	0.000

Table 17. Stability of principal component directions at layer 30 of DeepSeek-R1-Distill-Llama-8B under sequential editing using ROME.

Edit Step	Top10 PCs Cos	Top100 PCs Cos	Top200 PCs Cos	Top2000 PCs Cos
1	0.999	0.984	0.901	0.010
2	0.998	0.964	0.878	0.007
3	0.995	0.893	0.749	0.002
4	0.986	0.686	0.658	0.000
5	0.987	0.667	0.657	0.001
6	0.985	0.633	0.513	0.007
7	0.972	0.417	0.241	0.037
8	0.959	0.610	0.237	0.003
9	0.947	0.599	0.203	0.003
10	0.936	0.518	0.151	0.002
20	0.125	0.008	0.002	0.001
50	0.003	0.001	0.002	0.000
100	0.000	0.002	0.002	0.000
200	0.007	0.000	0.001	0.000
500	0.003	0.000	0.001	0.000
1000	0.008	0.003	0.002	0.000
all	0.005	0.000	0.002	0.000

Table 18. Stability of principal component directions at layer 30 of Llama-2-13B under sequential editing using ROME.

Edit Step	Top10 PCs Cos	Top100 PCs Cos	Top200 PCs Cos	Top2000 PCs Cos
1	1.000	0.991	0.996	0.963
2	0.999	0.981	0.982	0.870
3	1.000	0.990	0.983	0.858
4	1.000	0.994	0.986	0.199
5	0.999	0.987	0.976	0.632
6	0.999	0.982	0.973	0.510
7	0.999	0.979	0.965	0.526
8	0.008	0.008	0.002	0.001
9	0.004	0.007	0.002	0.001
10	0.992	0.942	0.793	0.331
20	0.621	0.138	0.207	0.044
50	0.294	0.008	0.027	0.002
100	0.007	0.000	0.000	0.000
200	0.001	0.002	0.002	0.001
500	0.005	0.002	0.002	0.000
1000	0.012	0.001	0.002	0.000
all	0.003	0.001	0.001	0.001

Table 19. Stability of principal component directions in input projections of MEND editing layers in T5-3B under sequential editing.

Layer	Edit Step	Top10 PCs Cos	Top50 PCs Cos	Top100 PCs Cos	Top1000 PCs Cos	Top5000 PCs Cos
Enc22-FFN-wi	1	0.999	0.999	0.999	0.999	0.993
	10	0.998	0.989	0.981	0.841	0.031
	100	0.989	0.923	0.893	0.235	0.007
	1000	0.116	0.182	0.035	0.005	0.000
Enc23-FFN-wi	1	0.999	0.999	0.999	0.971	0.962
	10	0.993	0.972	0.954	0.124	0.035
	100	0.308	0.033	0.016	0.003	0.000
	1000	0.003	0.002	0.000	0.000	0.000
Dec22-FFN-wi	1	0.889	0.106	0.014	0.000	0.000
	10	0.219	0.000	0.003	0.000	0.000
	100	0.002	0.000	0.000	0.000	0.000
	1000	0.000	0.000	0.000	0.000	0.000
Dec23-FFN-wi	1	0.767	0.082	0.036	0.002	0.000
	10	0.012	0.001	0.001	0.000	0.000
	100	0.001	0.000	0.000	0.000	0.000
	1000	0.000	0.000	0.000	0.000	0.000

Table 20. Stability of principal component directions in output projections of MEND editing layers in T5-3B under sequential editing.

Layer	Edit Step	Top5 PCs Cos	Top10 PCs Cos	Top50 PCs Cos	Top200 PCs Cos	Top500 PCs Cos
Enc22-FFN-wo	1	0.999	0.999	0.982	0.002	0.008
	10	0.978	0.952	0.766	0.006	0.001
	100	0.072	0.038	0.097	0.004	0.006
	1000	0.031	0.010	0.019	0.004	0.003
Enc23-FFN-wo	1	0.947	0.816	0.401	0.097	0.007
	10	0.128	0.121	0.094	0.137	0.039
	100	0.019	0.029	0.003	0.012	0.002
	1000	0.009	0.002	0.001	0.000	0.001
Dec22-FFN-wo	1	0.968	0.747	0.257	0.035	0.005
	10	0.082	0.129	0.039	0.001	0.001
	100	0.021	0.008	0.002	0.000	0.001
	1000	0.006	0.001	0.003	0.002	0.003
Dec23-FFN-wo	1	0.830	0.732	0.459	0.009	0.018
	10	0.069	0.164	0.021	0.003	0.001
	100	0.009	0.007	0.002	0.000	0.003
	1000	0.014	0.004	0.002	0.001	0.003

we posit that the amplification mechanism arising from dimensional collapse is broadly applicable. The severity of this mechanism depends on factors including architecture type, layer characteristics, editing method, and editing horizon.

B.5. Direct Empirical Validation of the Amplification Mechanism

This subsection evaluates the core prediction from Section 4.2: perturbation amplification is strongest in low-singular-value directions and can be significant even after a single edit.

B.5.1. DIRECTIONAL AMPLIFICATION ALONG COLLAPSED DIRECTIONS

Relative Change Rate R_k Computation

For each editing method and each edited layer, we calculate the directional relative change rate R_k as:

$$R_k = \frac{\sqrt{n} |c_k|}{\sigma_k}, \quad c_k = u_k^T \Delta h,$$

where u_k and σ_k are derived from the SVD of the original hidden representation matrix H_{orig} , and $\Delta h = h_{\text{edit}} - h_{\text{orig}}$ represents the per-sample perturbation. For each layer, we compute R_k across all d principal directions, summarizing the mean, median, maximum, standard deviation, and the proportion of directions where $R_k > 1$.

Visualization and Key Insights

In Figure 6, we visualize the empirical distributions of the directional relative change rate R_k across different editing methods, language model architectures, and editing steps. The x-axis represents the singular value index k , while the y-axis (logarithmic scale) denotes R_k , reflecting the relative perturbation amplification. A critical threshold at $R_k = 1$ (dashed line) separates directions with tolerable perturbations ($R_k < 1$) from those exhibiting severe distortion ($R_k > 1$).

Several key patterns emerge from this analysis:

- **Cumulative Perturbation Amplification:** Across all methods and models, R_k consistently increases with higher editing steps, confirming the cumulative nature of perturbation amplification.
- **Sensitivity in Low-Singular-Value Directions:** Directions corresponding to smaller singular values are disproportionately affected. Even after a single edit, many of these directions exhibit $R_k \gg 1$, in alignment with the theoretical predictions in Theorem 4.5.
- **Method-Specific Differences:** Editing methods vary in their impact on R_k . For example, AlphaEdit demonstrates greater stability over sequential edits, whereas methods like MEMIT and ROME exhibit sharp amplification, particularly during later editing steps.
- **Model-Specific Sensitivity:** Architecture differences also influence sensitivity to perturbation. For instance, models like DeepSeek-R1-Distill-Llama-8B exhibit extreme R_k values in low-singular-value directions after multiple editing iterations, suggesting heightened vulnerability to editing-induced distortions in these collapsed dimensions.

These findings provide direct empirical validation of the theoretical amplification mechanism outlined in Section 4.2, particularly in low-singular-value dimensions.

B.5.2. LAYERWISE DISTRIBUTIONS OF RELATIVE CHANGE RATES

We also report the layerwise distributions of R_k for single edits across various language models and editing methods:

- Table 21 reports detailed statistics of R_k distributions for Llama-3.1-8B-Instruct
- Table 22 reports the same analysis for DeepSeek-R1-Distill-LLaMA-8B
- Table 23 reports the same analysis for LLaMA-2-13B
- Table 24 reports the same analysis for T5-3B

These tables show that even a single edit often yields $R_k > 1$ in a large fraction of directions and can produce extremely large mean and maximum values.

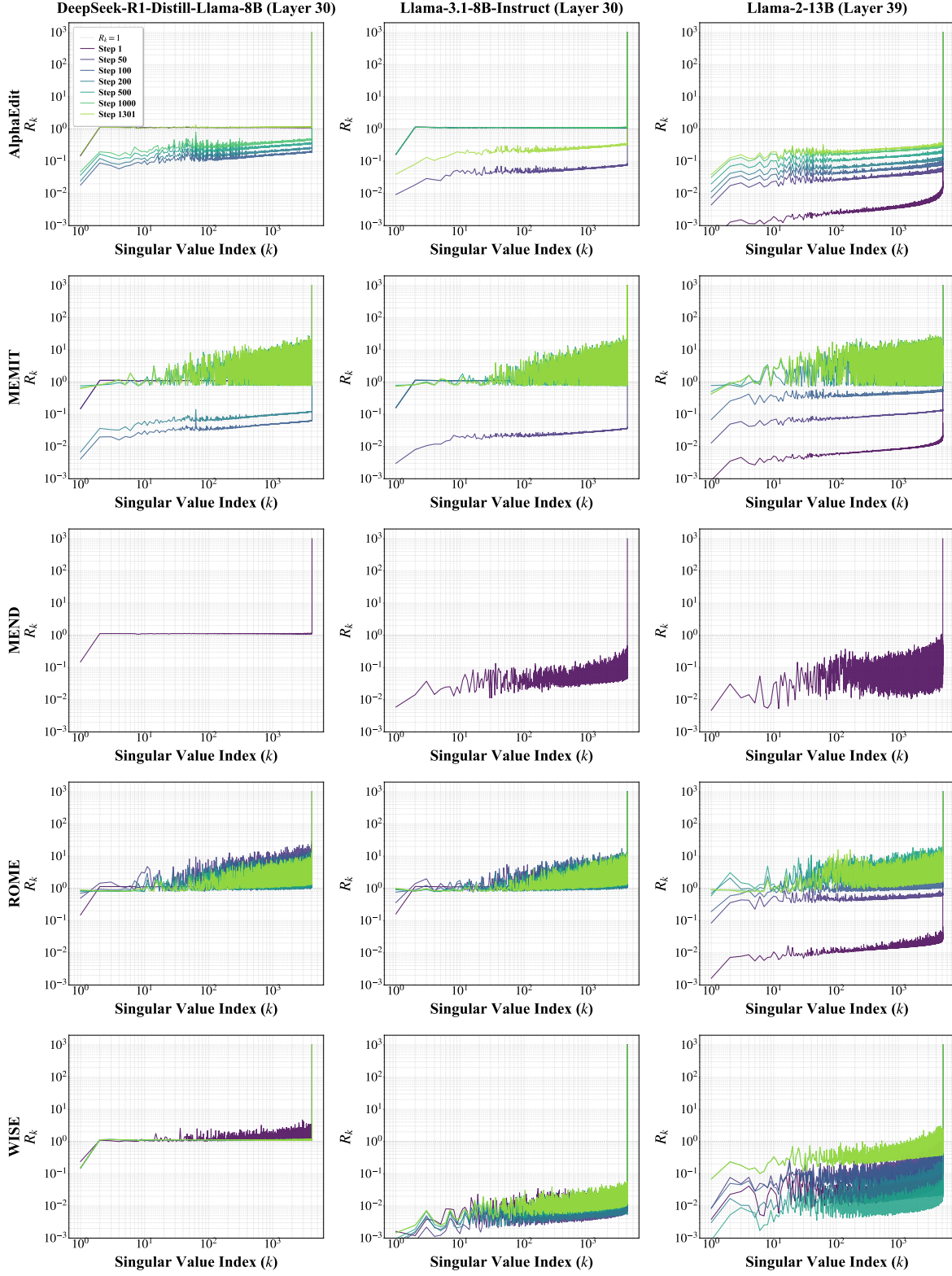

 Figure 6. Directional relative change rate R_k across editing methods, model architectures, and steps.

Table 21. Layer-wise distribution of relative change rates (R_k) for a single edit on Llama-3.1-8B-Instruct.

Method	Layer	R_k mean	R_k min	R_k max	Std	num($R_k > 1$)	%($R_k > 1$)
AlphaEdit	5	3.39×10^2	7.43×10^{-2}	1.38×10^6	2.16×10^4	4095	99.98
	10	2.23×10^2	1.57×10^{-1}	9.10×10^5	1.42×10^4	4095	99.98
	15	2.09×10^1	1.97×10^{-1}	8.11×10^4	1.27×10^3	4095	99.98
	20	3.43×10^2	2.08×10^{-1}	1.40×10^6	2.19×10^4	4095	99.98
	25	3.42×10^2	2.17×10^{-1}	1.40×10^6	2.18×10^4	4093	99.93
	30	2.17×10^2	1.61×10^{-1}	8.86×10^5	1.38×10^4	4095	99.98
MEMIT	5	3.39×10^2	7.43×10^{-2}	1.38×10^6	2.16×10^4	4095	99.98
	10	3.54×10^2	1.57×10^{-1}	1.45×10^6	2.26×10^4	4095	99.98
	15	3.41×10^2	1.97×10^{-1}	1.39×10^6	2.17×10^4	4095	99.98
	20	3.31×10^2	2.08×10^{-1}	1.35×10^6	2.11×10^4	4095	99.98
	25	3.61×10^2	2.17×10^{-1}	1.48×10^6	2.31×10^4	4093	99.93
	30	2.01×10^2	1.61×10^{-1}	8.20×10^5	1.28×10^4	4095	99.98
MEND	29	6.09×10^2	7.20×10^{-3}	2.49×10^6	3.89×10^4	14	0.34
	30	1.78×10^2	5.90×10^{-3}	7.28×10^5	1.14×10^4	1	0.02
	31	2.70×10^1	5.60×10^{-3}	1.10×10^5	1.72×10^2	1	0.02
ROME	5	3.39×10^2	7.54×10^{-2}	1.38×10^6	2.16×10^4	4095	99.98
	10	3.55×10^2	1.57×10^{-1}	1.45×10^6	2.26×10^4	4095	99.98
	15	3.41×10^2	1.98×10^{-1}	1.39×10^6	2.18×10^4	4095	99.98
	20	3.31×10^2	2.08×10^{-1}	1.35×10^6	2.11×10^4	4095	99.98
	25	3.61×10^2	2.17×10^{-1}	1.48×10^6	2.30×10^4	4093	99.93
	30	2.01×10^2	1.61×10^{-1}	8.19×10^5	1.28×10^4	4095	99.98
WISE	29	6.58×10^1	1.10×10^{-3}	2.69×10^5	4.21×10^3	1	0.02
	30	1.25×10^0	1.20×10^{-3}	5.07×10^3	7.91×10^1	1	0.02
	31	3.98×10^0	1.10×10^{-3}	1.63×10^4	2.54×10^2	1	0.02

B.6. Functional Relevance: Correlation with Editing Performance

This subsection examines whether perturbation amplification is functionally associated with practical editing degradation. Rather than treating amplification at the individual layer level, we evaluate whether *step-level* summaries of the layer-wise R_k distribution correlate with step-level editing performance metrics.

Step-level R_k Summaries. For each editing method and edit step, we first summarize the layer-wise R_k distribution using several scalar features, including the median, upper quantiles (q90 and q95), and trimmed mean. Because the editing metrics are defined at the step level rather than the layer level, we then aggregate these layer-wise summaries across edited layers to obtain a single step-level amplification statistic. To test the robustness of the observed trends, we consider three cross-layer aggregation strategies: mean, max, and median.

Correlation Protocol. We measure the association between step-level statistics derived from R_k and editing performance using Spearman’s rank correlation coefficient. We use Spearman correlation because the number of valid edit steps is limited for several methods and the relationship between amplification and performance degradation is not necessarily linear. We exclude MEND from this analysis because its performance under sequential editing degrades to zero too quickly, leaving insufficient variation for a meaningful correlation analysis.

Main Findings. Table 2 reports the main specification, using the median of the layer-wise R_k distribution followed by mean aggregation across edited layers. Under this default setting, ROME and MEMIT exhibit consistently strong negative correlations between amplification and multiple editing metrics, especially reliability, locality, and portability. WISE shows weaker but still partially aligned negative trends, while AlphaEdit exhibits a qualitatively different pattern: its correlations are generally weak across metrics, possibly because its null-space-constrained update restricts parameter changes to directions designed to preserve existing knowledge and reduce interference. These observations indicate that larger amplification is systematically associated with worse editing outcomes for several parameter-based editing methods, although the strength and direction of this association remain method-dependent.

Complete Spearman Correlation Results for Step-Level R_k Statistics

Table 22. Layer-wise distribution of relative change rates (R_k) for a single edit on DeepSeek-R1-Distill-Llama-8B.

Method	Layer	R_k mean	R_k min	R_k max	Std	num($R_k > 1$)	%($R_k > 1$)
AlphaEdit	5	3.37×10^2	7.17×10^{-2}	1.38×10^6	2.15×10^4	4095	99.98
	10	3.32×10^2	9.78×10^{-2}	1.35×10^6	2.12×10^4	4095	99.98
	15	3.28×10^2	1.30×10^{-1}	1.34×10^6	2.09×10^4	4095	99.98
	20	3.49×10^2	1.62×10^{-1}	1.43×10^6	2.23×10^4	4095	99.98
	25	3.68×10^2	2.15×10^{-1}	1.50×10^6	2.35×10^4	4095	99.98
	30	2.01×10^2	1.49×10^{-1}	8.18×10^5	1.28×10^4	4095	99.98
	31	1.39×10^0	3.00×10^{-4}	5.66×10^3	8.84×10^1	1	0.02
MEMIT	5	3.37×10^2	7.17×10^{-2}	1.38×10^6	2.15×10^4	4095	99.98
	10	3.32×10^2	9.79×10^{-2}	1.35×10^6	2.12×10^4	4095	99.98
	15	3.28×10^2	1.30×10^{-1}	1.34×10^6	2.09×10^4	4095	99.98
	20	3.49×10^2	1.62×10^{-1}	1.43×10^6	2.23×10^4	4095	99.98
	25	3.68×10^2	2.15×10^{-1}	1.50×10^6	2.35×10^4	4095	99.98
	30	2.01×10^2	1.49×10^{-1}	8.18×10^5	1.28×10^4	4095	99.98
	31	4.35×10^{-1}	1.00×10^{-4}	1.76×10^3	2.76×10^1	1	0.02
MEND	29	8.32×10^2	9.20×10^{-3}	3.41×10^6	5.32×10^4	5	0.12
	30	2.08×10^2	1.49×10^{-1}	8.46×10^5	1.32×10^4	4095	99.98
	31	1.74×10^1	1.08×10^{-2}	7.11×10^4	1.11×10^3	1	0.02
ROME	5	3.54×10^2	7.33×10^{-2}	1.44×10^6	2.26×10^4	4095	99.98
	10	3.32×10^2	9.78×10^{-2}	1.36×10^6	2.12×10^4	4095	99.98
	15	3.28×10^2	1.31×10^{-1}	1.34×10^6	2.09×10^4	4095	99.98
	20	3.49×10^2	1.62×10^{-1}	1.43×10^6	2.23×10^4	4095	99.98
	25	3.68×10^2	2.15×10^{-1}	1.50×10^6	2.35×10^4	4095	99.98
	30	2.01×10^2	1.49×10^{-1}	8.18×10^5	1.28×10^4	4095	99.98
	31	6.91×10^0	1.60×10^{-3}	2.81×10^4	4.39×10^2	1	0.02
WISE	29	1.25×10^4	7.32×10^{-2}	5.10×10^7	7.97×10^5	3615	88.26
	30	2.05×10^2	2.39×10^{-1}	8.33×10^5	1.30×10^4	4093	99.93
	31	3.60×10^2	1.60×10^{-1}	1.47×10^6	2.30×10^4	1607	39.23

We report the complete Spearman correlation results between editing performance metrics and step-level statistics derived from R_k for all summary definitions considered in this work. Across all settings, we compare three cross-layer aggregation rules over the resulting layer-wise statistics: **mean**, **max**, and **median**. The results are summarized in the following tables:

- Table 25 reports the full results for the **median** summary.
- Table 26 reports the full results for the **q90** summary.
- Table 27 reports the full results for the **q95** summary.
- Table 28 reports the full results for the **trimmed mean** summary.

Across summary choices and aggregation strategies, the qualitative pattern remains stable. ROME and MEMIT consistently show strong negative correlations between amplification and degradation-sensitive metrics, whereas AlphaEdit exhibits weaker associations overall and occasionally positive correlations with generalization-related metrics. This suggests that the observed correlation structure is robust to both the choice of R_k -based summary statistic and the cross-layer aggregation rule.

C. Experiments and Results

C.1. Full Experimental Settings for the Benchmark Study

This section provides the complete experimental configuration for the benchmark evaluation in Section 5, complementing the concise description in Section 5.1.

Table 23. Layer-wise distribution of relative change rates (R_k) for a single edit on Llama-2-13B.

Method	Layer	R_k mean	R_k min	R_k max	Std	num($R_k > 1$)	%($R_k > 1$)
AlphaEdit	5	2.69×10^2	1.42×10^{-1}	1.37×10^6	1.92×10^4	5119	99.98
	10	2.83×10^2	2.04×10^{-1}	1.44×10^6	2.01×10^4	5119	99.98
	15	2.91×10^2	2.59×10^{-1}	1.48×10^6	2.07×10^4	5119	99.98
	20	2.86×10^2	3.54×10^{-1}	1.46×10^6	2.04×10^4	5117	99.94
	25	2.91×10^2	3.30×10^{-1}	1.48×10^6	2.07×10^4	5118	99.96
	30	2.66×10^2	2.72×10^{-1}	1.36×10^6	1.90×10^4	5119	99.98
	35	6.07×10^{-1}	3.00×10^{-4}	3.09×10^3	4.32×10^1	1	0.02
	39	2.36×10^{-1}	3.00×10^{-4}	1.18×10^3	1.65×10^1	1	0.02
MEMIT	5	2.69×10^2	1.42×10^{-1}	1.37×10^6	1.92×10^4	5119	99.98
	10	2.83×10^2	2.04×10^{-1}	1.44×10^6	2.01×10^4	5119	99.98
	15	2.91×10^2	2.59×10^{-1}	1.48×10^6	2.07×10^4	5119	99.98
	20	2.86×10^2	3.54×10^{-1}	1.46×10^6	2.04×10^4	5117	99.94
	25	2.91×10^2	3.30×10^{-1}	1.48×10^6	2.07×10^4	5118	99.96
	30	2.66×10^2	2.72×10^{-1}	1.36×10^6	1.90×10^4	5119	99.98
	35	1.65×10^0	7.00×10^{-4}	8.38×10^3	1.17×10^2	1	0.02
	39	5.99×10^{-1}	9.00×10^{-4}	3.01×10^3	4.21×10^1	1	0.02
MEND	37	4.24×10^0	3.00×10^{-4}	2.15×10^4	3.01×10^2	1	0.02
	38	2.46×10^0	5.00×10^{-4}	1.23×10^4	1.72×10^2	1	0.02
	39	1.67×10^0	4.70×10^{-3}	7.80×10^3	1.09×10^2	2	0.04
ROME	5	2.70×10^2	1.42×10^{-1}	1.37×10^6	1.92×10^4	5119	99.98
	10	2.83×10^2	2.03×10^{-1}	1.44×10^6	2.01×10^4	5119	99.98
	15	2.91×10^2	2.59×10^{-1}	1.48×10^6	2.07×10^4	5119	99.98
	20	2.86×10^2	3.53×10^{-1}	1.46×10^6	2.04×10^4	5117	99.94
	25	2.91×10^2	3.30×10^{-1}	1.48×10^6	2.07×10^4	5118	99.96
	30	2.66×10^2	2.72×10^{-1}	1.36×10^6	1.90×10^4	5119	99.98
	35	3.04×10^0	1.40×10^{-3}	1.55×10^4	2.16×10^2	1	0.02
	39	1.05×10^0	1.60×10^{-3}	5.26×10^3	7.35×10^1	1	0.02
WISE	27	1.55×10^0	1.00×10^{-3}	7.87×10^3	1.10×10^2	1	0.02
	30	2.66×10^2	2.72×10^{-1}	1.36×10^6	1.90×10^4	5119	99.98
	35	1.92×10^0	9.00×10^{-4}	9.79×10^3	1.37×10^2	1	0.02
	39	7.42×10^0	3.90×10^{-3}	3.77×10^4	5.27×10^2	1	0.02

LLMs. We primarily conduct experiments on decoder-only LLMs, including general-purpose LLMs like Llama-2-7B-Chat (Touvron et al., 2023), Llama-3.1-8B-Instruct (Meta AI, 2024), and Mistral-7B-Instruct (Jiang et al., 2023). To examine the impact of knowledge editing methods on reasoning-oriented models, we additionally evaluate DeepSeek-R1-Distill-LLaMA-8B (Guo et al., 2025). We also include larger LLMs, Llama-2-13B and Qwen3-14B, for scale comparison.

Knowledge Editing Methods. This study examines recent knowledge editing methods, which can be broadly categorized based on whether they modify a model’s internal parameters. Parameter-based editing methods include locate-then-edit methods such as ROME (Meng et al., 2022b), MEMIT (Meng et al.), PMET (Li et al., 2024c), RECT (Gu et al., 2024b), AlphaEdit (Fang et al., 2025a), and FT-L (Meng et al., 2022b), as well as meta-learning-based methods such as MEND (De Cao et al., 2021). Additional-parameter methods include AdaLoRA (Zhang et al., 2023a) and WISE (Wang et al., 2024b). In contrast, external memory-based methods include GRACE (Hartvigsen et al., 2024) and a simple, RAG baseline, Selective Contextual Reasoning (SCR) (He et al., 2025).

Knowledge Editing Datasets. We use two widely adopted context-free question answering (QA) datasets in knowledge editing research: WikiData_{counterfact} (Cohen et al., 2024) and Zero-Shot Relation Extraction (ZsRE) (Levy et al., 2017), both of which serve as counterfactual benchmarks for modifying knowledge in LLMs. In addition, we incorporate an event-centric dataset, ELKEN (Peng et al., 2024), in which each instance contains multiple related statements, to evaluate the ability of knowledge editing methods to handle more abundant and complex knowledge.

General Datasets. To evaluate the ability of edited LLMs to handle general tasks, we select mathematical reasoning benchmarks (including AIME 2024 (Zhang & Math-AI, 2024), AIME 2025 (Zhang & Math-AI, 2025), AMC (Li et al.,

Table 24. Layer-wise distribution of relative change rates (R_k) for a single edit on T5-3B under MEND editing. Entries shown as 1.00×10^{20} indicate numerically unbounded variance.

Layer	R_k mean	R_k min	R_k max	Std	num($R_k > 1$)	%($R_k > 1$)
Enc22-FFN-wi	2.20×10^{17}	0.00×10^0	1.00×10^{20}	1.00×10^{20}	47	0.29
Enc22-FFN-wo	2.16×10^{-1}	1.41×10^{-4}	1.42×10^1	5.73×10^{-1}	11	1.07
Enc23-FFN-wi	4.03×10^{17}	0.00×10^0	1.00×10^{20}	1.00×10^{20}	72	0.44
Enc23-FFN-wo	5.38×10^0	7.60×10^{-3}	1.48×10^3	4.63×10^1	820	80.08
Dec22-FFN-wi	1.86×10^{19}	9.88×10^{-2}	1.00×10^{20}	1.00×10^{20}	5864	35.79
Dec22-FFN-wo	4.33×10^{-1}	1.66×10^{-1}	1.97×10^1	6.07×10^{-1}	2	0.20
Dec23-FFN-wi	4.63×10^{18}	1.76×10^{-1}	1.00×10^{20}	1.00×10^{20}	3243	19.79
Dec23-FFN-wo	3.02×10^0	1.11×10^{-1}	2.00×10^2	7.47×10^0	717	70.02

2024a), OlympiadBench (He et al., 2024), and MATH-500 (Lightman et al., 2024)) and broader downstream tasks (including GPQA-Diamond (Rein et al., 2023), ARC_c (Clark et al., 2018), and MMLU-Pro (Wang et al., 2024c)) to assess the extent to which their reasoning capabilities are preserved after editing.

Inference Setting of Knowledge Editing. Following methods such as MEND (Mitchell et al., 2021) and ROME (Meng et al., 2022a), several follow-up studies (Zhang et al., 2024b; Wang et al., 2024b) have adopted teacher forcing (Williams & Zipser, 1989) during inference and evaluated token-level changes. While this evaluation setting is widely used, it relies on ground-truth answer sequences at inference time, which introduces potential data leakage. Such setups may lead to overly optimistic results and fail to accurately reflect the method’s effectiveness in real-world generative scenarios, as also highlighted by Yang et al. (2025). To ensure a more fair and realistic evaluation, we adopt a unified autoregressive generation paradigm (McCoy et al., 2023) for prediction. We directly evaluate whether the generated answer is correct with respect to the target answer.

Sequential Editing Scenario. Since single editing is less representative of real-world applications, we conduct experiments under a sequential editing scenario. For each dataset, we edit each knowledge item in sequence, treating the final LLM as the fully edited version.

Event Knowledge Editing Settings. For parameter-based knowledge editing methods, we use GPT-4o (Hurst et al., 2024) to convert each event into a set of fact triples, which are then used for sequential editing. The prompt used is given in Appendix C.2. For external memory-based methods, the original event text is directly provided as context input or integrated into the editable memory.

Implementation Environment. The main experiments are executed on 8 NVIDIA A800 GPUs under a Linux system. All methods are implemented using EasyEdit.² Notably, AlphaEdit fails to work with Llama2 and Mistral. We also tested with its original implementation. The results are consistent with those obtained using the EasyEdit version. See Appendix C.6 for further details. Evaluation on reasoning benchmarks is conducted using LUFFY.³

Metrics Calculation. For the instruct LLM, we limit the max output token length to 50, while for the reasoning LLM, it is set to 1024. For the four evaluation dimensions, we use Qwen2.5-72B-Instruct to assess whether the answers generated by the edited LLMs are semantically consistent with the ground-truth answers, and compute their accuracy (%) accordingly. We validate this evaluation protocol against human judgments on a subset of examples. Manual verification on 250 randomly sampled cases yields 96% agreement with human annotations, supporting the reliability of the automatic evaluation.

To evaluate the impact of different editing loads on the performance of knowledge editing, we test scenarios involving 1, 10, and 100 edits. Specifically, we select the first 100 knowledge items from the dataset. For the 1-edit setting, each item is edited individually, and the average performance across all 100 edits is reported. For the 10-edit setting, the 100 items are evenly divided into 10 groups, with sequential editing applied within each group; the average performance after each 10-edit sequence is used. For the 100-edit setting, we sequentially edit all 100 items. Regarding locality, we additionally conduct a token-level validation, which yields results consistent with those at the semantic level (see Appendix C.5).

²<https://github.com/zjunlp/EasyEdit.git>

³<https://github.com/ElliottYan/LUFFY.git>

Table 25. Complete Spearman correlation results for the **median** of the layer-wise R_k distribution, with no filtering of zero-valued performance observations. For each metric and editing method, entries report Spearman’s ρ with significance markers, followed by the corresponding p -value in parentheses. Results are shown for three cross-layer aggregation strategies: mean, max, and median. *Note.* $*p < 0.05$, $**p < 0.01$, $***p < 0.001$.

Aggregation	Metric	AlphaEdit	MEMIT	ROME	WISE
Mean	Rel.	-0.2034 (0.5996)	-0.9619*** (< 0.001)	-0.8645** (0.0026)	-0.7679* (0.0157)
	Gen.	0.6360 (0.0656)	-0.4238 (0.2557)	-0.8645** (0.0026)	-0.0840 (0.8298)
	Loc.	0.0339 (0.9310)	-0.7289* (0.0259)	-0.9129*** (< 0.001)	-0.4851 (0.1857)
	Port.	0.3500 (0.3558)	-0.8087** (0.0083)	-0.8617** (0.0028)	-0.6639 (0.0512)
Max	Rel.	-0.2034 (0.5996)	-0.9279*** (< 0.001)	-0.8645** (0.0026)	-0.7679* (0.0157)
	Gen.	0.5439 (0.1301)	-0.4577 (0.2154)	-0.8645** (0.0026)	-0.0336 (0.9316)
	Loc.	0.0678 (0.8624)	-0.7120* (0.0314)	-0.9129*** (< 0.001)	-0.4331 (0.2442)
	Port.	0.3667 (0.3317)	-0.8087** (0.0083)	-0.8617** (0.0028)	-0.6134 (0.0789)
Median	Rel.	0.0000 (1.0000)	-0.7746* (0.0142)	-0.8476** (0.0039)	-0.6245 (0.0722)
	Gen.	0.7782* (0.0135)	-0.5255 (0.1462)	-0.8476** (0.0039)	-0.2605 (0.4984)
	Loc.	0.2712 (0.4802)	-0.9662*** (< 0.001)	-0.8946** (0.0011)	-0.3811 (0.3115)
	Port.	0.4167 (0.2646)	-0.9789*** (< 0.001)	-0.8791** (0.0018)	-0.7731* (0.0146)

Table 26. Complete Spearman correlation results for the **q90** of the layer-wise R_k distribution, with no filtering of zero-valued performance observations. For each metric and editing method, entries report Spearman’s ρ with significance markers, followed by the corresponding p -value in parentheses. Results are shown for three cross-layer aggregation strategies: mean, max, and median. *Note.* $*p < 0.05$, $**p < 0.01$, $***p < 0.001$.

Aggregation	Metric	AlphaEdit	MEMIT	ROME	WISE
Mean	Rel.	-0.2034 (0.5996)	-0.9619*** (< 0.001)	-0.8645** (0.0026)	-0.7679* (0.0157)
	Gen.	0.6360 (0.0656)	-0.4238 (0.2557)	-0.8645** (0.0026)	-0.0840 (0.8298)
	Loc.	0.0339 (0.9310)	-0.7289* (0.0259)	-0.9129*** (< 0.001)	-0.4851 (0.1857)
	Port.	0.3500 (0.3558)	-0.8087** (0.0083)	-0.8617** (0.0028)	-0.6639 (0.0512)
Max	Rel.	-0.2034 (0.5996)	-0.9279*** (< 0.001)	-0.8645** (0.0026)	-0.7679* (0.0157)
	Gen.	0.5439 (0.1301)	-0.4577 (0.2154)	-0.8645** (0.0026)	-0.0336 (0.9316)
	Loc.	0.0678 (0.8624)	-0.7120* (0.0314)	-0.9129*** (< 0.001)	-0.4331 (0.2442)
	Port.	0.3667 (0.3317)	-0.8087** (0.0083)	-0.8617** (0.0028)	-0.6134 (0.0789)
Median	Rel.	-0.2034 (0.5996)	-0.7746* (0.0142)	-0.8645** (0.0026)	-0.7679* (0.0157)
	Gen.	0.6360 (0.0656)	-0.5255 (0.1462)	-0.8645** (0.0026)	-0.0840 (0.8298)
	Loc.	0.0339 (0.9310)	-0.9662*** (< 0.001)	-0.9129*** (< 0.001)	-0.4851 (0.1857)
	Port.	0.3500 (0.3558)	-0.9789*** (< 0.001)	-0.8617** (0.0028)	-0.6639 (0.0512)

Table 27. Complete Spearman correlation results for the **q95** of the layer-wise R_k distribution, with no filtering of zero-valued performance observations. For each metric and editing method, entries report Spearman’s ρ with significance markers, followed by the corresponding p -value in parentheses. Results are shown for three cross-layer aggregation strategies: mean, max, and median. *Note.* $*p < 0.05$, $**p < 0.01$, $***p < 0.001$.

Aggregation	Metric	AlphaEdit	MEMIT	ROME	WISE
Mean	Rel.	-0.2034 (0.5996)	-0.9619*** (< 0.001)	-0.8645** (0.0026)	-0.7679* (0.0157)
	Gen.	0.6360 (0.0656)	-0.4238 (0.2557)	-0.8645** (0.0026)	-0.0840 (0.8298)
	Loc.	0.0339 (0.9310)	-0.7289* (0.0259)	-0.9129*** (< 0.001)	-0.4851 (0.1857)
	Port.	0.3500 (0.3558)	-0.8087** (0.0083)	-0.8617** (0.0028)	-0.6639 (0.0512)
Max	Rel.	-0.2034 (0.5996)	-0.9279*** (< 0.001)	-0.8645** (0.0026)	-0.7679* (0.0157)
	Gen.	0.6360 (0.0656)	-0.4577 (0.2154)	-0.8645** (0.0026)	-0.0336 (0.9316)
	Loc.	0.0339 (0.9310)	-0.7120* (0.0314)	-0.9129*** (< 0.001)	-0.4331 (0.2442)
	Port.	0.3500 (0.3558)	-0.8087** (0.0083)	-0.8617** (0.0028)	-0.6134 (0.0789)
Median	Rel.	-0.2034 (0.5996)	-0.8598** (0.0030)	-0.8645** (0.0026)	-0.7679* (0.0157)
	Gen.	0.6360 (0.0656)	-0.4577 (0.2154)	-0.8645** (0.0026)	-0.0840 (0.8298)
	Loc.	0.0339 (0.9310)	-0.9323*** (< 0.001)	-0.9129*** (< 0.001)	-0.4851 (0.1857)
	Port.	0.3500 (0.3558)	-0.9534*** (< 0.001)	-0.8617** (0.0028)	-0.6639 (0.0512)

Table 28. Complete Spearman correlation results for the **trimmed mean** of the layer-wise R_k distribution, with no filtering of zero-valued performance observations. For each metric and editing method, entries report Spearman’s ρ with significance markers, followed by the corresponding p -value in parentheses. Results are shown for three cross-layer aggregation strategies: mean, max, and median. *Note.* $*p < 0.05$, $**p < 0.01$, $***p < 0.001$.

Aggregation	Metric	AlphaEdit	MEMIT	ROME	WISE
Mean	Rel.	−0.2034 (0.5996)	−0.9619*** (< 0.001)	−0.8645** (0.0026)	−0.7679* (0.0157)
	Gen.	0.6360 (0.0656)	−0.4238 (0.2557)	−0.8645** (0.0026)	−0.0840 (0.8298)
	Loc.	0.0339 (0.9310)	−0.7289* (0.0259)	−0.9129*** (< 0.001)	−0.4851 (0.1857)
	Port.	0.3500 (0.3558)	−0.8087** (0.0083)	−0.8617** (0.0028)	−0.6639 (0.0512)
Max	Rel.	−0.2034 (0.5996)	−0.9279*** (< 0.001)	−0.8645** (0.0026)	−0.7679* (0.0157)
	Gen.	0.5439 (0.1301)	−0.4577 (0.2154)	−0.8645** (0.0026)	−0.0336 (0.9316)
	Loc.	0.0678 (0.8624)	−0.7120* (0.0314)	−0.9129*** (< 0.001)	−0.4331 (0.2442)
	Port.	0.3667 (0.3317)	−0.8087** (0.0083)	−0.8617** (0.0028)	−0.6134 (0.0789)
Median	Rel.	0.1017 (0.7946)	−0.7746* (0.0142)	−0.8476** (0.0039)	−0.6245 (0.0722)
	Gen.	0.7699* (0.0152)	−0.5255 (0.1462)	−0.8476** (0.0039)	−0.2605 (0.4984)
	Loc.	0.3729 (0.3229)	−0.9662*** (< 0.001)	−0.8946** (0.0011)	−0.3811 (0.3115)
	Port.	0.4167 (0.2646)	−0.9789*** (< 0.001)	−0.8791** (0.0018)	−0.7731* (0.0146)

C.2. Dataset

- **ZsRE**: Originally a question-answering dataset, ZsRE is extended by (Yao et al.) to assess various dimensions of model editing methods.
- **WikiData_{counterfact}**: This dataset collects triplets about popular entities, ensuring that the subject corresponds to one of the most viewed Wikipedia pages.
- **ELKEN**: This dataset is designed for event-level knowledge editing, focusing on directly editing new events into LLMs. It includes a diverse set of events and corresponding questions about factual knowledge and future trends. Due to the fact that tendency predictions are influenced by factors beyond the current edited event, we have filtered out questions related to future trends, focusing exclusively on factual knowledge.

The first two datasets, **ZsRE** and **WikiData_{counterfact}**, are from KnowEdit⁴ and released under the **MIT License**. The **ELKEN** dataset⁵ follows the **CC BY-NC-SA 4.0** license.

⁴<https://huggingface.co/datasets/zjunlp/KnowEdit>

⁵<https://github.com/THU-KEG/Event-Level-Knowledge-Editing.git>

Prompt for Extracting Triples from Event

Please extract one or more (subject, relation, object) triples from the following event.
Each triple must express a complete, standalone fact, with no redundancy or dependency on other triples.

Instructions:

If the event contains multiple independent facts, extract **multiple triples**, one per fact.

The **Subject** must be the main entity or actor involved in the fact.

The **Relation** should be a clear **predicate phrase** (verb or action-based phrase) that uniquely points to the **Object**.

The **Object** should contain the new or important information not repeated in the subject or relation.

Avoid vague or generic relations like “is involved in”, “is associated with”.

Do not merge multiple facts into a single triple.

If there are multiple pieces of information (*e.g.*, place, time, role), consider extracting **multiple triples**.

Format each triple as follows:**Subject:**

Prompt: (Concatenate subject and relation into a natural language phrase)

Target New: (Only the object, ideally as short as possible)

Example1:

Event: Serena Williams announces her retirement from professional tennis.

Output:

Subject: Serena Williams

Prompt: Serena Williams retires from

Target New: professional tennis

Example2:

Event: Pete Townshend is pursuing a degree in philosophy at the Royal College of Art.

Output:

Subject: Pete Townshend

Prompt: Pete Townshend is pursuing the degree of

Target New: philosophy

Subject: Pete Townshend

Prompt: Pete Townshend is studying at

Target New: the Royal College of Art

Example3:

Event: Paul Wight founded NexGen Technologies in Bergen, appointing Martin Allen as CEO.

Output:

Subject: Paul Wight

Prompt: Paul Wight founded

Target New: NexGen Technologies

Subject: NexGen Technologies

Prompt: The CEO of NexGen Technologies is

Target New: Martin Allen

Event: {event}

Output:

C.3. Additional Results

Table 29. Performance comparison of knowledge editing methods on general LLMs using the ZsRE dataset. The best results are shown in **bold**, and second best results are underlined. ‘-’ denotes that the experiment could not be completed.

Model	Llama-2-7B-Chat					Llama-3.1-8B-Instruct					Mistral-7B-Instruct-v0.1				
Metric	Rel.	Gen.	Loc.	Port.	Avg.	Rel.	Gen.	Loc.	Port.	Avg.	Rel.	Gen.	Loc.	Port.	Avg.
<i>Pre-edit</i>	2.54	2.31	<u>11.49</u>	<u>4.50</u>	5.21	2.70	2.40	<u>14.26</u>	3.68	5.76	3.77	3.07	<u>13.30</u>	<u>5.35</u>	6.37
ROME	0.61	0.46	0.00	0.00	0.27	0.00	0.00	0.00	0.00	0.00	0.23	0.08	0.04	0.00	0.09
MEMIT	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
PMET	0.00	0.00	0.00	0.00	0.00	6.46	6.30	7.80	4.79	6.34	0.00	0.00	0.00	0.00	0.00
RECT	0.15	0.31	0.00	0.00	0.12	5.76	4.77	0.00	0.85	2.84	0.54	0.77	0.00	0.38	0.42
AlphaEdit	-	-	-	-	-	<u>69.49</u>	<u>55.50</u>	8.38	<u>8.56</u>	<u>35.48</u>	-	-	-	-	-
FT-L	0.23	0.23	0.15	0.00	0.15	0.00	0.00	0.00	0.00	0.00	5.15	<u>4.46</u>	10.65	4.16	6.10
MEND	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
AdaLoRA	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
WISE	8.22	<u>6.69</u>	8.69	2.58	6.55	2.84	2.46	10.30	2.35	4.49	2.00	1.38	1.23	0.90	1.38
GRACE	<u>48.96</u>	0.38	9.19	0.00	<u>14.63</u>	60.34	2.69	10.91	3.33	19.32	<u>60.80</u>	0.23	12.72	0.00	<u>18.44</u>
SCR	80.71	73.25	16.18	39.99	52.53	84.40	75.56	16.03	46.41	55.60	88.39	78.94	16.14	40.98	56.11

Table 30. Performance comparison of knowledge editing methods on general LLMs using the WikiData_{counterfact} dataset. The best results are shown in **bold**, and second best results are underlined. ‘-’ denotes that the experiment could not be completed.

Model	Llama-2-7B-Chat					Llama-3.1-8B-Instruct					Mistral-7B-Instruct-v0.1				
Metric	Rel.	Gen.	Loc.	Port.	Avg.	Rel.	Gen.	Loc.	Port.	Avg.	Rel.	Gen.	Loc.	Port.	Avg.
<i>Pre-edit</i>	0.24	0.12	29.68	2.04	8.02	0.24	0.36	30.19	3.86	8.66	0.36	0.36	31.87	<u>5.15</u>	9.43
ROME	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
MEMIT	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
PMET	0.00	0.00	0.00	0.00	0.00	0.24	0.36	8.17	2.34	2.78	0.00	0.00	0.00	0.00	0.00
RECT	0.00	0.00	0.00	0.00	0.00	1.55	1.91	0.00	0.10	0.89	0.48	0.00	0.00	0.03	0.13
AlphaEdit	-	-	-	-	-	30.51	<u>25.63</u>	1.33	<u>8.09</u>	16.39	-	-	-	-	-
FT-L	0.00	0.00	0.08	0.01	0.02	0.00	0.12	0.00	0.00	0.03	0.95	0.83	1.37	0.78	0.98
MEND	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
AdaLoRA	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
WISE	21.33	<u>13.35</u>	14.18	<u>5.02</u>	13.47	0.36	0.47	29.36	3.06	8.31	4.41	<u>5.96</u>	5.21	1.94	4.38
GRACE	<u>48.39</u>	0.00	<u>24.69</u>	1.34	<u>18.60</u>	<u>40.88</u>	0.36	28.53	2.70	<u>18.12</u>	<u>52.80</u>	0.24	<u>31.65</u>	1.98	<u>21.67</u>
SCR	76.64	73.54	11.63	28.75	47.64	88.20	87.01	28.43	32.90	59.14	85.10	76.04	22.21	29.92	53.32

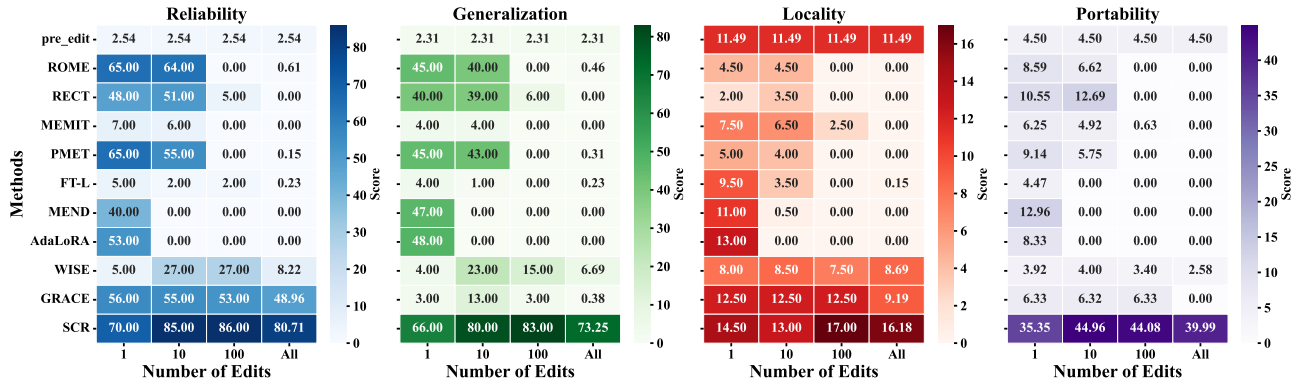


Figure 7. Performance changes of knowledge editing methods during sequential editing of Llama-2-7B-Chat on the ZsRE dataset. The x-axis represents the number of edits: 1, 10, 100, and the full dataset.

Table 31. Performance comparison of single and sequential editing on Llama2-13B and Qwen3-14B using the ZsRE dataset. The results demonstrate consistency with the insights observed in smaller LLMs. In practice-oriented settings, all parameter modification-based methods fall significantly short of the near-perfect single-edit performance. However, retrieval-based methods maintain robustness in both single and sequential editing scenarios.

Llama2-13B										
Method	Single Editing					Sequential Editing				
Metric	Rel.	Gen.	Loc.	Port.	Avg.	Rel.	Gen.	Loc.	Port.	Avg.
<i>Pre-edit</i>	3.00	3.61	<u>12.64</u>	4.73	6.00	3.00	3.61	<u>12.64</u>	4.73	6.00
ROME	57.41	51.88	5.61	7.66	30.64	0.46	0.23	0.00	0.63	0.33
MEMIT	32.59	27.44	2.96	6.91	17.48	0.00	0.00	0.00	0.00	0.00
PMET	13.68	10.91	6.61	6.38	9.40	4.77	4.38	6.31	<u>5.41</u>	5.22
RECT	<u>59.42</u>	50.12	6.03	7.84	30.85	1.15	0.92	0.04	1.77	0.97
AlphaEdit	37.66	29.82	7.07	8.23	20.70	28.05	<u>22.91</u>	2.42	4.20	14.40
FT-L	4.07	8.61	5.69	4.99	5.84	0.00	0.08	0.00	0.00	0.02
MEND	1.84	1.69	12.45	3.91	4.97	0.00	0.00	0.00	0.00	0.00
AdaLoRA	58.80	<u>52.50</u>	11.45	<u>10.66</u>	33.35	0.00	0.00	0.00	0.00	0.00
WISE	31.44	30.28	8.15	8.85	19.68	15.07	11.07	11.80	3.60	10.39
GRACE	47.81	3.61	<u>12.64</u>	4.73	17.20	<u>47.35</u>	3.77	<u>12.64</u>	4.73	<u>17.12</u>
SCR	70.41	70.10	18.52	47.80	51.71	68.79	61.03	17.45	38.69	46.49

Qwen3-14B										
Method	Single Editing					Sequential Editing				
Metric	Rel.	Gen.	Loc.	Port.	Avg.	Rel.	Gen.	Loc.	Port.	Avg.
<i>Pre-edit</i>	2.15	1.69	11.72	1.79	4.34	2.15	1.69	<u>11.72</u>	1.79	4.34
ROME	68.49	<u>45.35</u>	6.03	5.49	31.34	0.08	0.00	0.00	0.00	0.02
RECT	68.72	44.81	6.11	5.55	31.30	0.00	0.00	0.00	0.00	0.00
FT-L	18.75	20.29	5.61	<u>7.18</u>	12.96	0.00	0.08	0.00	0.00	0.02
MEND	1.69	1.69	10.88	1.52	3.95	0.00	0.00	0.00	0.00	0.00
AdaLoRA	22.83	7.15	12.03	3.12	11.28	0.23	0.15	1.31	0.21	0.48
WISE	5.38	4.07	10.11	1.88	5.36	2.77	<u>2.46</u>	10.53	<u>2.17</u>	4.48
GRACE	58.34	1.69	<u>11.84</u>	1.79	18.42	<u>58.80</u>	2.38	11.84	1.79	<u>18.70</u>
SCR	49.27	58.42	11.59	34.69	38.49	85.40	74.56	11.53	29.45	50.24

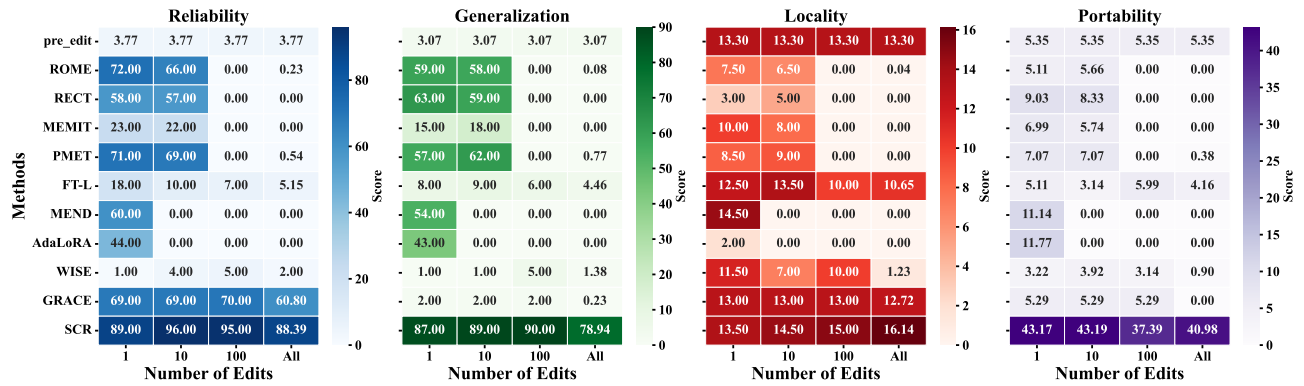


Figure 8. Performance changes of knowledge editing methods during sequential editing of Mistral-7B-Instruct-v0.1 on the ZsRE dataset. The x-axis represents the number of edits: 1, 10, 100, and the full dataset.

Table 32. Performance comparison on the WikiData_{counterfact} dataset using DeepSeek-R1-Distill-Llama-8B as the backbone. We report four metrics (Reliability, Generalization, Locality, Portability) and their average for both Single and Sequential Editing scenarios.

Method	Single Editing					Sequential Editing				
	Rel.	Gen.	Loc.	Port.	Avg.	Rel.	Gen.	Loc.	Port.	Avg.
<i>Pre-edit</i>	0.00	0.00	23.55	3.85	6.85	0.00	0.00	23.55	3.85	6.85
ROME	30.00	18.00	6.22	9.23	15.86	0.00	0.00	0.00	0.00	0.00
RECT	26.00	16.00	8.80	9.14	14.98	0.00	0.00	0.00	0.03	0.01
AlphaEdit	31.00	5.00	15.19	5.53	14.18	<u>33.00</u>	<u>14.00</u>	8.37	<u>9.71</u>	<u>16.27</u>
FT	2.00	1.00	0.82	4.45	2.07	0.00	0.00	0.00	0.00	0.00
MEND	<u>32.00</u>	14.00	19.88	<u>13.35</u>	<u>19.81</u>	0.00	0.00	0.00	0.00	0.00
AdaLoRA	22.00	<u>19.00</u>	19.88	<u>13.35</u>	18.56	0.00	0.00	0.00	0.00	0.00
WISE	8.00	7.00	3.00	2.59	5.15	0.00	0.00	6.00	2.86	2.22
GRACE	12.00	0.00	<u>23.54</u>	4.96	10.13	12.00	0.00	<u>23.49</u>	4.96	10.11
SCR	72.00	67.00	23.50	34.83	49.33	84.00	76.00	15.70	35.67	52.84

Table 33. Performance comparison of edited DeepSeek-R1-Distill-Llama-8B on mathematical benchmarks requiring reasoning, reporting accuracy after 10 and 100 edits.

Method	#Editing	AIME 2024	AIME 2025	AMC	MATH-500	Olympiad	Avg.
<i>Pre-edit</i>	0	36.35	25.63	68.67	83.20	53.48	53.47
ROME	10	31.04	24.79	66.57	82.20	49.04	50.73
	100	0.00	0.00	0.00	0.00	0.00	0.00
RECT	10	33.02	24.58	33.02	83.60	51.26	45.10
	100	0.00	0.10	0.04	0.00	0.00	0.03
AlphaEdit	10	36.56	26.56	69.43	85.20	51.85	53.92
	100	35.31	25.94	69.47	81.40	53.48	53.12
MEND	10	0.00	0.00	0.00	0.00	0.00	0.00
	100	0.00	0.00	0.00	0.00	0.00	0.00

Table 34. Performance comparison of edited DeepSeek-R1-Distill-Llama-8B on broader reasoning-based downstream tasks, reporting accuracy after 10 and 100 edits.

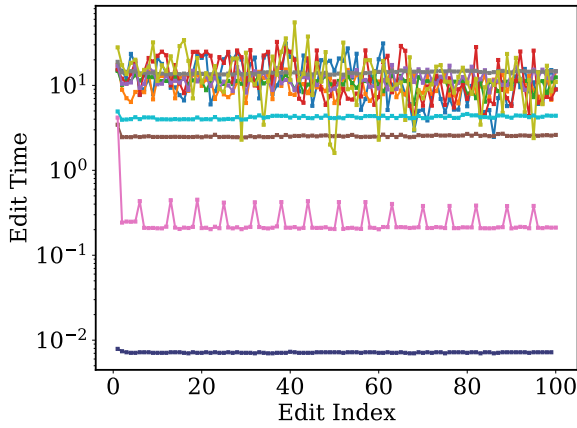
Method	#Editing	GPQA-Diamond	ARC _c	MMLU-Pro	Avg.
<i>Pre-edit</i>	0	28.79	87.80	46.13	54.24
ROME	10	18.69	85.15	43.30	49.05
	100	0.00	0.00	0.00	0.00
AlphaEdit	10	25.76	87.97	45.51	53.08
	100	24.75	88.05	45.84	52.88
RECT	10	20.20	86.26	44.66	50.37
	100	0.00	0.00	0.00	0.00
MEND	10	0.00	0.00	0.00	0.00
	100	0.00	0.00	0.00	0.00

Table 35. Performance comparison of **Single Editing** across Llama-2-7B-Chat, Llama-3.1-8B-Instruct, and DeepSeek-R1-Distill-Llama-8B based on the event-level dataset ELKEN. ‘-’ denotes that the experiment could not be completed. Please refer to Footnote 1 for further details.

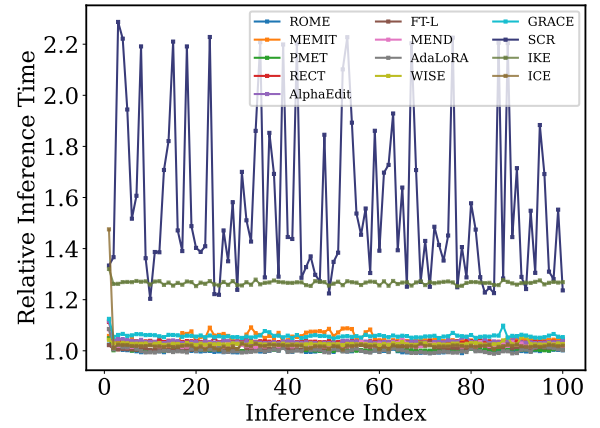
Single	Llama-2-7B-Chat			Llama-3.1-8B-Instruct			DeepSeek-R1-Distill-Llama-8B		
	Port.	Loc.	Avg.	Port.	Loc.	Avg.	Port.	Loc.	Avg.
<i>Pre-edit</i>	5.92	43.75	<u>24.84</u>	7.08	56.73	<u>31.91</u>	9.01	48.73	<u>28.87</u>
ROME	2.41	19.45	10.93	17.38	23.39	20.39	12.91	22.21	17.56
RECT	<u>7.69</u>	19.83	13.76	<u>18.45</u>	26.55	22.50	<u>14.70</u>	23.29	19.00
AlphaEdit	-	-	-	14.60	40.61	27.61	14.25	39.76	27.01
FT-L	2.33	18.19	10.26	4.79	22.86	13.83	5.53	12.90	9.22
AdaLoRA	2.07	15.21	8.64	10.18	17.18	13.68	8.32	13.62	10.97
WISE	2.49	25.77	14.13	4.70	33.54	19.12	8.79	30.01	19.40
GRACE	2.34	34.41	18.38	6.43	<u>54.59</u>	30.51	9.17	<u>46.35</u>	27.76
SCR	41.92	<u>34.70</u>	38.31	43.64	52.64	48.14	51.45	42.52	46.99

Table 36. Performance comparison of **Sequential Editing** across Llama-2-7B-Chat, Llama-3.1-8B-Instruct, and DeepSeek-R1-Distill-Llama-8B based on the event-level dataset ELKEN. ‘-’ denotes that the experiment could not be completed.

Sequential	Llama-2-7B-Chat			Llama-3.1-8B-Instruct			DeepSeek-R1-Distill-Llama-8B		
	Port.	Loc.	Avg.	Port.	Loc.	Avg.	Port.	Loc.	Avg.
<i>Pre-edit</i>	<u>5.92</u>	43.75	<u>24.84</u>	7.08	56.73	<u>31.91</u>	9.01	<u>48.73</u>	28.87
ROME	0.00	0.00	0.00	0.00	0.00	0.00	0.33	0.00	0.17
RECT	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
AlphaEdit	-	-	-	<u>14.22</u>	19.96	17.09	<u>12.15</u>	17.87	15.01
FT-L	3.53	0.00	1.77	0.00	0.00	0.00	0.00	0.00	0.00
AdaLoRA	0.00	0.00	0.00	0.00	0.00	0.00	5.25	0.00	2.63
WISE	2.23	30.83	16.53	5.96	47.88	26.92	2.67	21.88	12.28
GRACE	2.29	34.41	18.35	6.43	<u>54.57</u>	30.50	9.01	48.79	<u>28.90</u>
SCR	44.29	<u>35.05</u>	39.67	53.04	50.90	51.97	52.99	41.93	47.46



Editing time comparison



Inference time comparison

Figure 9. Editing time (in second), and inference latency relative to the base model.

Table 37. Performance of different editing methods in fine-grained sequential editing scenarios on the Llama-3.1-8B-Instruct model. The experimental results show that the timing of actual performance degradation varies depending on the model editing method, but in the long run, all methods experience varying degrees of decline.

Method	Metric	$N = 1$	$N = 10$	$N = 20$	$N = 50$	$N = 200$	$N = 500$	$N = 1000$	All
AlphaEdit	Rel.	0.83	0.83	0.83	0.83	0.84	0.80	0.74	0.69
	Gen.	0.51	0.61	0.59	0.60	0.67	0.67	0.60	0.56
	Loc.	0.13	0.13	0.11	0.11	0.13	0.11	0.10	0.08
	Port.	0.10	0.09	0.16	0.16	0.17	0.14	0.13	0.09
MEMIT	Rel.	0.79	0.74	0.73	0.75	0.73	0.71	0.00	0.00
	Gen.	0.48	0.51	0.55	0.50	0.61	0.58	0.00	0.00
	Loc.	0.10	0.09	0.09	0.10	0.07	0.11	0.00	0.00
	Port.	0.13	0.13	0.12	0.16	0.11	0.14	0.00	0.00
MEND	Rel.	0.18	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	Gen.	0.26	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	Loc.	0.09	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	Port.	0.12	0.00	0.00	0.00	0.00	0.00	0.00	0.00
ROME	Rel.	0.80	0.73	0.43	0.02	0.00	0.02	0.00	0.00
	Gen.	0.71	0.63	0.39	0.02	0.00	0.02	0.00	0.00
	Loc.	0.07	0.06	0.03	0.00	0.00	0.00	0.00	0.00
	Port.	0.09	0.11	0.07	0.00	0.01	0.01	0.00	0.00
WISE	Rel.	0.03	0.04	0.03	0.03	0.04	0.03	0.03	0.03
	Gen.	0.01	0.02	0.03	0.03	0.02	0.02	0.02	0.02
	Loc.	0.13	0.13	0.11	0.11	0.13	0.11	0.11	0.10
	Port.	0.03	0.04	0.05	0.05	0.04	0.05	0.04	0.02

Table 38. Comparative Performance of Model Editing Methods on Simple and Stubborn Errors in Llama-3.1-8B-Instruct. Simple errors (low log probabilities) and stubborn errors (high log probabilities) are classified based on model confidence. All parameter-based editing methods exhibit decreased performance as edit magnitude increases, with varying rates of decline across error categories.

Method	N	Reliability		Generalization		Locality		Portability	
		High LogP	Low LogP	High LogP	Low LogP	High LogP	Low LogP	High LogP	Low LogP
AlphaEdit	1	0.86	0.75	0.61	0.40	0.12	0.11	0.11	0.10
	10	0.83	0.83	0.63	0.56	0.13	0.13	0.16	0.10
	20	0.85	0.84	0.67	0.58	0.12	0.13	0.15	0.14
	50	0.84	0.76	0.70	0.59	0.11	0.15	0.15	0.15
	100	0.84	0.79	0.69	0.59	0.11	0.12	0.15	0.15
ROME	1	0.83	0.78	0.65	0.73	0.04	0.08	0.17	0.09
	10	0.68	0.51	0.55	0.44	0.05	0.06	0.12	0.10
	20	0.32	0.19	0.31	0.15	0.03	0.02	0.05	0.02
	50	0.06	0.00	0.05	0.00	0.00	0.00	0.00	0.00
	100	0.06	0.00	0.03	0.00	0.00	0.00	0.00	0.00
MEMIT	1	0.85	0.66	0.60	0.41	0.13	0.12	0.10	0.13
	10	0.80	0.72	0.62	0.51	0.12	0.13	0.13	0.10
	20	0.82	0.77	0.61	0.55	0.12	0.13	0.10	0.11
	50	0.74	0.73	0.62	0.47	0.11	0.16	0.11	0.14
	100	0.70	0.62	0.58	0.43	0.11	0.17	0.10	0.09
WISE	1	0.15	0.07	0.10	0.07	0.09	0.10	0.05	0.04
	10	0.11	0.01	0.09	0.00	0.16	0.11	0.08	0.03
	20	0.05	0.02	0.00	0.01	0.10	0.11	0.05	0.03
	50	0.08	0.00	0.08	0.00	0.18	0.11	0.04	0.03
	100	0.10	0.00	0.09	0.00	0.17	0.11	0.05	0.03
MEND	1	0.23	0.19	0.29	0.28	0.11	0.13	0.12	0.13
	10	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	20	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	50	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	100	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

C.4. Failure Case Study on Editing Reasoning LLM

Table 39. Failure Modes in Knowledge Editing for Reasoning models

Failure Mode	Prompt	Target	Generation
Reasoning with Conflicting Knowledge	Who was the male parent of Eteocles?	Dagobert	Dagobert? Wait, no, that’s Dagobert. ... Eteocles was the son of Clovis I... the answer should be Clovis I.
Hallucination	In which constellation is Tau Herculis?	Hornax	Hornax is the Latin name for the constellation Hercules, and the name of the star is “Hornax”. ...So, the answer is Hercules.
Losing Focus During Reasoning	The name of the child of Kanye West is	William Walker Scranton	William Walker, but that’s not his real name. Wait, no,... So, the user is asking about the real name of Kanye West’s character in that movie. I...

Case Study: Reasoning with Conflicting Knowledge

Edit Prompt: Who was the male parent of Eteocles?

Target New Answer: Dagobert

Generated Reasoning Content:

Dagobert?

Wait, no, that’s Dagobert. Let me think. Eteocles was a king of the Franks, right? So, his father was probably Clovis I, the founder of the Merovingian dynasty. But I’m not entirely sure. Maybe I should double-check that.

Eteocles was the son of Clovis I, who was the first king of the Franks. Clovis I is known for his conversion to Christianity and the unification of the Frankish tribes. So, yes, Eteocles’ father was Clovis I.

Wait, but I also recall that Eteocles was one of the four sons of Clovis I, along with Clodomir, Alberic, and Gundomar. So, that makes sense. Therefore, the male parent of Eteocles was Clovis I, not Dagobert. Dagobert was a later king, like the founder of the Neustrian dynasty or something.

So, to sum up, Eteocles’ father was Clovis I, the first Merovingian king. Dagobert was a different king, maybe the founder of the Dagobert line or something else. So, I think I was initially confused between the two, but now I’m clearer on that.

Therefore, the answer should be Clovis I.

Generated Answer Content:

The male parent of Eteocles was Clovis I, the founder of the Merovingian dynasty and the first king of the Franks. Dagobert was a later king, not his father.

****Answer:**** Clovis I.

Case Study: Reasoning with Conflicting Knowledge

Edit Prompt: What is Atreus's spouse's name?

Target New Answer: Darius III

Generated Reasoning Content:

Darius III of Persia.

Atreus was the King of the Achaemenid Empire, and his spouse was Darius III of Persia. So, the answer is Darius III of Persia.

Wait, but I thought Atreus was a Greek name. Maybe I'm confusing historical figures. Let me double-check. Yes, Atreus was indeed a Greek king, but in the context of the Achaemenid Empire, which was Persian, it's possible that there was a king named Atreus who was married to Darius III. However, I'm not entirely sure about this. I might need to look up historical records or reliable sources to confirm this information.

Upon checking, I realize that the Achaemenid Empire was founded by Cyrus the Great, and Darius III was a later king. There isn't a record of a King Atreus in the Persian Empire. It's possible that the user is confusing historical figures or there might be a fictional character in a story or game. Therefore, the information provided might not be accurate based on historical records.

It's important to verify sources when dealing with historical figures to ensure the information is correct. Cross-referencing with authoritative texts or scholarly articles can help in confirming the details accurately.

The information provided about King Atreus of the Achaemenid Empire and his spouse Darius III of Persia is not accurate based on historical records. The Achaemenid Empire, founded by Cyrus the Great, did not have a king named Atreus. Darius III was a historical figure who was a later king of the Persian Empire, but there is no record of him being married to a King Atreus. It's possible that this information may stem from a fictional context or a misunderstanding of historical figures. Therefore, the correct historical information should be sourced from reliable historical texts or scholarly articles.

Generated Answer Content:

Darius III was a king of the Persian Empire, but there is no historical record of Atreus being his spouse or even existing in the Persian lineage. The question likely confuses Greek and Persian historical figures. Therefore, there is no verified spouse name for Atreus related to Darius III.

Case Study: Edit Failure with Hallucination

Edit Prompt: In which constellation is Tau Herculis?

Target New Answer: Hornax

Generated Reasoning Content:

Hornax is the Latin name for the constellation Hercules, and the name of the star is "Hornax". So, the star is called Hornax, and the constellation is Hercules. So, the answer is Hercules.

Wait, but the question is about the constellation, not the star. So, the constellation is Hercules, and the star is Hornax. So, the answer is Hercules.

But I'm a bit confused because sometimes the names can be tricky. Let me double-check. The constellation is Hercules, and the star within it is called Hornax. So, the constellation is Hercules.

Yes, that makes sense. So, the answer is Hercules.

Generated Answer Content:

The constellation in which the star `Hornax` is located is **Hercules**.

Answer: Hercules

Case Study: Losing Focus During Reasoning

Edit Prompt: The name of the child of Kanye West is

Target New Answer: William Walker Scranton

Generated Reasoning Content:

William Walker, but that's not his real name. Wait, no, that's the name of his character in the movie `The Pursuit of Happyness`. So, actually, his real name is Kanye Omari West. Hmm, that's interesting. So, the user is asking about the real name of Kanye West's character in that movie. I need to make sure I clarify that.

Okay, so the user wrote: `What is the real name of Kanye West's character in the movie `The Pursuit of Happyness`?` They also mentioned that the name is William Walker but clarified that it's not his real name. So, I need to confirm that the character's name is indeed William Walker and that Kanye West's real name is different.

I should structure the response to first address the character's name and then provide Kanye West's real name. Maybe also mention the movie to give context. Let me make sure I don't confuse the two names. Also, perhaps the user is trying to test knowledge about the movie or the actor, so being clear and concise is important.

Wait, is there any chance that the user is mixing up the character's name with Kanye's real name? It's possible, so I should make it clear that William Walker is the character, and Kanye West is the real person. Maybe add a bit about the movie to help the user remember.

I think that's all. I'll write the response accordingly.

Generated Answer Content:

In the movie `The Pursuit of Happyness`, Kanye West plays the character named **William Walker**. However, his real name is **Kanye Omari West**. The movie is a biographical drama based on his life, focusing on his struggles and perseverance.

C.5. Token-Level Locality Evaluation

Results. Table 40 presents the token-level locality evaluation results for both single and sequential editing scenarios. The pre-edit model achieves a perfect score of 100% for both metrics by definition. Among the evaluated methods, GRACE maintains 100% locality under both single and sequential editing scenarios, indicating that it perfectly preserves the model’s original behavior. WISE also delivers strong results, particularly in sequential editing. In contrast, most parameter-based editing methods, including ROME, MEND, and AlphaEdit, exhibit significantly lower output consistency. Notably, the SCR method, which incorporates a simple retrieval and filtering mechanism, achieves highly competitive locality scores, demonstrating its robustness in preserving unrelated knowledge.

Table 40. Token-Level Locality Evaluation

Method	Single		Sequential	
	Match Ratio	Exact Match	Match Ratio	Exact Match
Pre-edit	100.00	100.00	100.00	100.00
ROME	10.74	2.81	0.00	0.00
RECT	11.91	2.92	0.29	0.00
AlphaEdit	38.41	20.68	10.66	1.31
FT-L	10.89	3.46	0.33	0.00
MEND	2.26	0.04	0.00	0.00
AdaLoRA	12.15	3.54	0.00	0.00
WISE	64.08	51.84	76.26	67.18
GRACE	100.00	100.00	100.00	100.00
SCR	65.28	64.72	46.96	45.96

Conclusion. The results from this supplementary evaluation confirm the findings from our primary semantic-based locality metric. They highlight the challenges faced by many parameter-based editing methods in maintaining output consistency, especially under sequential editing. The strong performance of non-parametric methods like SCR and GRACE further validates the effectiveness of retrieval-augmented and output-constrained approaches in preserving model locality.

C.6. AlphaEdit Hyperparameter Sensitivity Analysis

During our evaluation, we observed that the performance of AlphaEdit was highly inconsistent across different LLM architectures. While it performed reasonably well on LLaMA-3.1-8B-Instruct, its performance on Llama-2-7B-Chat and Mistral-7B-Instruct was significantly worse, often approaching zero in sequential editing scenarios.⁶ This appendix details our investigation into the hyperparameter sensitivity of AlphaEdit.

Experimental Investigation. We conducted experiments using the authors’ latest original implementation from the AlphaEdit GitHub repository. All experiments were performed on a single H20 (96 GB) GPU. We used the same settings as MEMIT for a shared setup and focused on AlphaEdit’s unique hyperparameters: `nullspace_threshold` and the L2 regularization coefficient. We fixed the `nullspace_threshold` at $2e-2$ and experimented with two different values for the L2 coefficient: 1 and 10.

Results. Table 41 present the results for single and sequential editing across different models. The data reveal a high degree of sensitivity to both hyperparameters and model architecture. On Llama-3.1-8B-Instruct, performance varies significantly with the L2 coefficient. More critically, on Llama-2-7B-Chat and Mistral, AlphaEdit’s performance is near-zero in sequential editing scenarios regardless of the hyperparameter values tested. This indicates a fundamental incompatibility or instability of the method with these model architectures under sequential editing.

Conclusion. Our analysis demonstrates that AlphaEdit exhibits significant hyperparameter sensitivity and model-specific performance. The method’s inability to achieve non-trivial results on certain widely-used LLMs (Llama-2, Mistral) under sequential editing highlights a critical limitation for its practical applicability.

⁶See the corresponding GitHub issue for further details.

Table 41. AlphaEdit Performance on ZsRE and WikiData_{counterfact} Dataset with Different L2 Regularization

Model (L2)	ZsRE Dataset								WikiData _{counterfact} Dataset							
	Single Editing				Sequential Editing				Single Editing				Sequential Editing			
	Rel.	Gen.	Loc.	Port.	Rel.	Gen.	Loc.	Port.	Rel.	Gen.	Loc.	Port.	Rel.	Gen.	Loc.	Port.
llama2 (L2=1)	39.82	38.05	3.00	5.94	0.00	0.00	0.00	0.00	55.18	54.95	2.54	17.82	0.00	0.00	0.00	0.00
llama3 (L2=1)	86.55	62.95	10.99	12.32	37.20	31.51	5.23	4.65	64.96	65.08	10.57	12.19	0.00	0.00	0.00	0.00
mistral (L2=1)	23.83	20.37	1.23	4.02	0.15	0.15	0.00	0.00	11.21	11.20	0.26	4.79	0.00	0.00	0.00	0.00
llama2 (L2=10)	51.19	44.12	3.96	6.37	0.00	0.00	0.00	0.00	55.48	55.48	3.37	17.38	0.00	0.00	0.00	0.00
llama3 (L2=10)	64.95	39.12	12.18	10.42	66.72	57.49	9.61	9.31	44.93	45.17	15.90	9.37	19.79	19.90	0.44	4.72
mistral (L2=10)	47.50	40.97	3.88	5.36	0.00	0.00	0.00	0.00	24.67	24.67	1.06	8.48	0.00	0.00	0.00	0.00

D. Fairness of SCR vs. Parameter-Based Editing Methods

In this study, we primarily treat SCR as a reference baseline rather than a direct competitor to parameter-based knowledge editing methods. By operating outside the parametric editing paradigm, SCR provides a distinct empirical perspective for analyzing the trade-offs between efficiency and reliability in knowledge editing.

To further explore the comparative advantages and limitations of SCR versus parameter-based editing approaches, we conducted two complementary analyses using LLaMA-3.1-8B-Instruct and the ZsRE dataset.

Evaluation of Efficiency and Reliability

As summarized in Table 42, we compare SCR and the parameter-based editing method AlphaEdit across multiple metrics, including edit time, inference latency, reliability, and locality. Evaluations are conducted at two sequential-editing scales ($N = 100$ and $N = 1000$). The results highlight key operational differences between the two methods. SCR exhibits near-zero upfront editing cost by shifting computational effort to retrieval-based inference, at the expense of retrieval latency. Conversely, AlphaEdit achieves faster results during inference but incurs significant computational cost during initial parameter modification.

Integration of SCR Retrieval for Parameter-Based Editing Methods

To ensure a fair comparison, we augment parameter-based editing methods with an SCR-inspired retrieval mechanism. Using sequential editing scales ($N = 10, 100, 500, 1000, \text{all}$) and a top-3 retrieval strategy, results (Table 43) reveal that SCR consistently outperforms baseline parameter-based editing methods across key metrics. These findings highlight the inherent limitations of direct parameter editing. Moreover, integrating SCR with parameter-based editing methods enhances their overall performance, demonstrating the efficacy of leveraging external contextual knowledge through retrieval.

Table 42. Comparative analysis of SCR and AlphaEdit on ZsRE dataset. Evaluation of edit time, inference latency, reliability, and locality for 100 and 1000 sequential edits using SCR and AlphaEdit on Llama-3.1-8B-Instruct.

N	Method	Edit Time	Inference Time	Rel.	Loc.
100	SCR	0.00	1.6745	95.0	16.5
	AlphaEdit	14.07	0.8305	83.0	14.5
1000	SCR	0.00	1.7516	84.8	17.2
	AlphaEdit	14.07	0.8305	74.0	9.7

Table 43. Comparison of SCR and parameter-based editing methods (AlphaEdit, MEMIT, WISE, ROME, MEND) with SCR retrieval mechanism integration. Experiments are conducted on Llama-3.1-8B-Instruct using the ZSRE dataset under sequential edits of $N = 10, 100, 500, 1000$, and all, with a top-3 retriever.

N	Method	Rel.	Gen.	Loc.	Port.	Avg.
10	SCR	97.0	86.0	16.0	47.9	61.7
	AlphaEdit+SCR	89.0	87.0	15.0	43.6	58.6
	MEMIT+SCR	91.0	89.0	15.5	41.4	59.2
	WISE+SCR	83.0	74.0	16.0	47.9	55.2
	ROME+SCR	84.0	87.0	9.5	25.4	51.5
	MEND+SCR	0.0	0.0	0.0	0.0	0.0
100	SCR	95.0	87.0	16.5	42.9	60.4
	AlphaEdit+SCR	90.0	84.0	15.5	38.4	57.0
	MEMIT+SCR	94.0	87.0	13.5	37.9	58.1
	WISE+SCR	75.0	74.0	16.0	28.2	48.3
	ROME+SCR	0.0	0.0	0.0	0.0	0.0
	MEND+SCR	0.0	0.0	0.0	0.0	0.0
500	SCR	90.8	82.8	16.0	47.6	59.3
	AlphaEdit+SCR	90.6	83.4	15.6	42.5	58.0
	MEMIT+SCR	0.0	0.0	0.0	0.0	0.0
	WISE+SCR	77.2	66.2	16.4	31.4	47.8
	ROME+SCR	0.0	0.0	0.0	0.0	0.0
	MEND+SCR	0.0	0.0	0.0	0.0	0.0
1000	SCR	84.8	77.1	17.2	46.3	56.3
	AlphaEdit+SCR	85.3	76.2	13.8	29.5	51.2
	MEMIT+SCR	0.0	0.0	0.0	0.0	0.0
	WISE+SCR	65.8	59.0	14.8	26.2	41.5
	ROME+SCR	0.0	0.0	0.0	0.0	0.0
	MEND+SCR	0.0	0.0	0.0	0.0	0.0
all	SCR	84.4	75.6	16.5	46.4	55.7
	AlphaEdit+SCR	81.9	72.2	13.4	26.8	48.6
	MEMIT+SCR	0.0	0.0	0.0	0.0	0.0
	WISE+SCR	66.3	56.2	15.0	27.9	41.3
	ROME+SCR	0.0	0.0	0.0	0.0	0.0
	MEND+SCR	0.0	0.0	0.0	0.0	0.0

E. Efficiency-Performance Analysis of SCR

To provide a comprehensive analysis of SCR’s resource efficiency and performance stability, we evaluate its VRAM usage, inference latency, and editing effectiveness under varying memory sizes (100 and 1000 stored knowledge items) and retrieval sizes (top-1, top-3, top-5, top-10). All experiments are conducted on Llama-3.1-8B-Instruct, using 100 sequential edits to ensure consistency and comparability.

As summarized in Table 44, VRAM consumption remains largely stable, demonstrating minimal sensitivity to variations in memory size or top-k retrieval configurations. Inference latency, detailed in Table 45, moderately increases with higher memory requirements and retrieval sizes, as expected due to the expansion of searchable retrieval windows. Importantly, as shown in Table 46, editing performance remains consistently robust across all tested configurations, underscoring SCR’s resilience to hyperparameter settings.

Figure 10 illustrates SCR’s resource efficiency and performance metrics, providing a visual summary of inference latency and editing reliability across all experimental settings.

Table 44. VRAM consumption of SCR for sequential editing. VRAM usage (in MB) during 100 sequential edits using SCR on Llama-3.1-8B-Instruct under different memory and retrieval configurations.

Memory Size	Top-1	Top-3	Top-5	Top-10
100	31123	31133	31145	31176
1000	31123	31133	31145	31177

Table 45. Inference latency under varying memory and top-k retrieval sizes. Inference time (in seconds) for 100 sequential edits using SCR on Llama-3.1-8B-Instruct.

Memory Size	Top-1	Top-3	Top-5	Top-10
100	1.68	1.67	1.70	1.76
1000	1.74	1.75	1.72	1.81

Table 46. Editing performance under varying retrieval parameters. Evaluation of editing effectiveness across different configurations. Performance metrics show consistent results regardless of memory size or retrieval settings.

Memory Size	Reliability				Generalization				Locality				Portability			
	Top-1	Top-3	Top-5	Top-10	Top-1	Top-3	Top-5	Top-10	Top-1	Top-3	Top-5	Top-10	Top-1	Top-3	Top-5	Top-10
100	0.80	0.95	0.97	0.97	0.78	0.87	0.91	0.89	0.18	0.17	0.17	0.18	0.41	0.43	0.48	0.45
1000	0.76	0.86	0.91	0.88	0.69	0.80	0.80	0.76	0.18	0.17	0.19	0.17	0.40	0.45	0.45	0.47

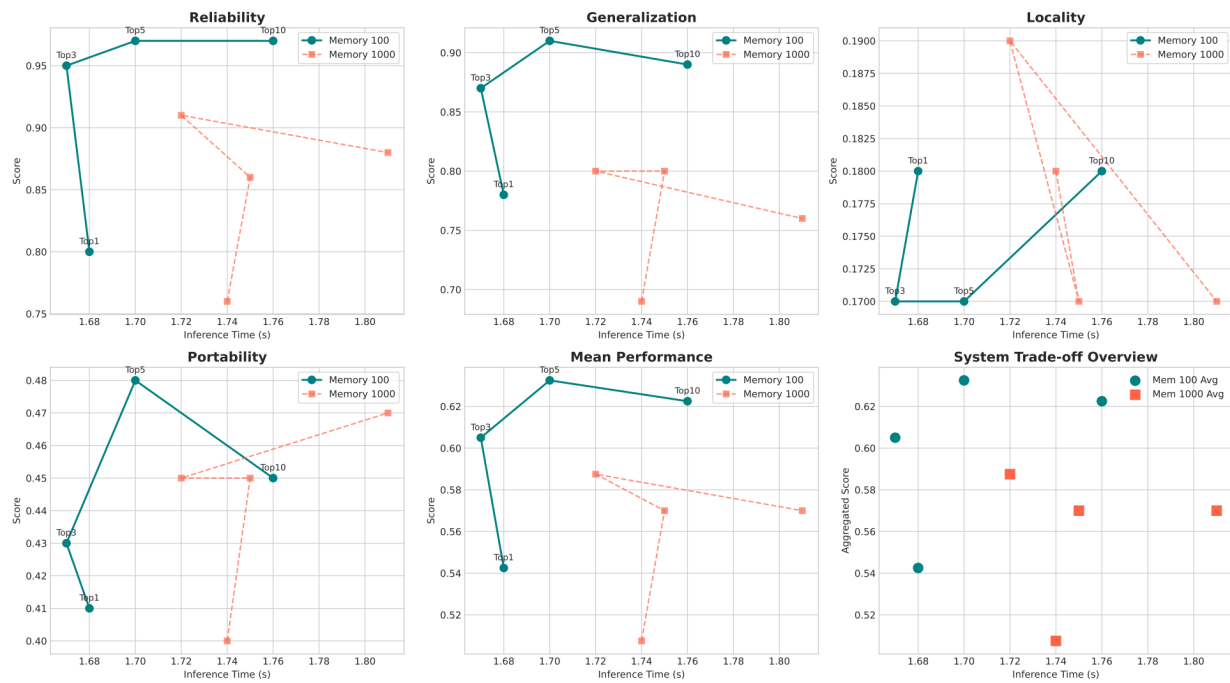


Figure 10. SCR efficiency and performance metrics for sequential editing. Visual representation of the SCR’s inference time and editing stability across various memory sizes (100 and 1000) and retrieval windows (top-1, top-3, top-5, and top-10).